

Language Is Not Transparent

An Exploratory Study of Semantic Density, Architecture, and
Constraint Design in LLMs

Author: Adam Givon

Date: 14.6.2026

Archive: <https://doi.org/10.5281/zenodo.20689806>

All GitHub paths are relative to: <https://github.com/adamgivon/language-is-not-transparent>

Table of Contents

Preface.....	1
Chapter One: Theory.....	2
1. The Thesis.....	2
2. Related Work and Positioning.....	3
3. The Systems.....	4
4. Design Principles.....	10
5. Mechanisms: How Constraints Operate.....	12
6. Principles vs. Rules.....	19
7. Implementation.....	20
Chapter Two: The Experiment.....	23
1. Experiment Setup.....	23
2. Analytical Procedures.....	25
3. Evaluation Procedures.....	27
4. Results - Numerical Statistics.....	31
5. Results - Close Reading.....	36
6. The Systems' Architecture and Its Influence: Architecture as a Map.....	54
Acknowledgement.....	58
Annex: GitHub Repository Structure.....	59

Preface

I have been working on this project for a long time now - longer than I planned or wanted. There were moments of crisis along the way - directions taken that fell through, and methods that failed to advance the work and had to be discarded. Now it is done.

This paper is a continuation of two earlier works. The first, *The Anchored Form*, described the anchoring system itself - what it is, why it was created, and how. The second, written some months later as Chapter 0 of that paper's revised edition, came after both the system and I had matured. This second paper drew the boundaries, named the failure modes and the dangers, and preserved the original text untouched as a record of where the work began. This third paper is broader and more empirical in method. It no longer studies one anchoring system but four, and uses them as samples through which to examine the relations between constraints, language, form, governance, and human-AI communication in general.

In hindsight, the three papers seem to form a bridge between two worlds: ours and that of AI. Each paper approaches this bridge from a different angle, and together they form a trilogy. The trajectory the three papers form is the part I most want the reader to see. The first was written looking from the inside out, and its limits were therefore blurry. It already held what later became cleaner directions - form over memory, principles over rules, constraint as a stabilizing force - but it accepted the models' own descriptions of themselves, creating a problematic circularity. The second changed the angle to an outside look, and drew the borders: what the framework can hold, where it fails, what it makes possible for better and for worse. This third paper steps further out and measures. It replaces the models' testimony with evidence - distances, comparisons, close reading of outputs. Each paper broadens the angle, looking for a wider mechanism and simpler common ground, and on the way, retires the kind of evidence the one before it leaned on. If the trilogy shows anything, it is a method correcting its own way of knowing, in public.

What this last paper argues is that a constraint should not only instruct the model what to do. Through its language and its structure, it should demonstrate how to do it. Written this way, a constraint becomes both the direction and the pattern.

I recently saw, out of the corner of my eye, some news about humanistic AI systems proposed at Microsoft. The first paper reached for that same perspective before I knew anyone would give it a name, and I think the humanistic perspective is the one thing needed above all else. The dual nature of constraints - direction and pattern at once - could let us write AI constitutions, if they are written correctly. I hope they will be.

I hope you enjoy the paper. I also recommend reading the first two works - not because this one depends on them, but because they show where it came from, and perspective matters.

Thank you,

Adam Givon

Chapter One: Theory

1. The Thesis

The main thesis of this paper is that constraint writing for LLM systems is achieved through three distinct, non-interchangeable mechanisms: explicit logic, semantic density and language architecture. Logic specifies what the constraint does - its operational content and targets. Semantic density determines how strongly the constraint binds behavior. Language architecture determines how constraints are structured, organized, and related to each other. These three are non-interchangeable: the same logic, expressed with different density and architecture, will yield different behavior. A constraint expressed in semantically dense language will be different from a similar constraint expressed in flat procedural language, even when both are supported by explicit rules. The important thing to remember is that it is necessary to match the conceptual and functional purposes of the constraint to the language and architecture used. A simple task will likely require simple language and structure. A complicated task or a heavily conceptual constraint will likely need richer language that addresses multiple concepts and includes multiple framing sub-rules, thus creating a more sophisticated structure alongside more complex language.

Let me expand it a bit further. Current practice uses language as a carrying device, as a transparent layer through which logic is transferred to the model. My claim is that the actual relationship is different. Language is constitutive and is not transparent - it does not only carry the logic, it is part of the constraint and affects how strongly the logic binds. It has its own internal properties that can enforce the logic but also distort it. Language has its own internal constraints embedded in words. It is not merely a vehicle, it is also a passenger that comes with its own luggage. To put it another way: in human systems, enforcement comes from context, consequences, and social pressure. In LLM systems, language is not merely an interface layer. It is the recurring governance medium through which behavior is shaped across training, alignment, prompting, evaluation, and constraint design. Thus, the choice of expression is elevated from an aesthetic preference to an operational necessity. In a way, writing constraints might not be about instructing the LLM; it might actually be about re-wiring the model's reasoning with each new prompt. The perception shifts: what I, as a human, call a constraint, the model receives as a change in semantic topography. The intended effect is the same, but the mechanism is very different.

Language fundamentally shapes the model's cognitive landscape. Semantic richness or sparsity, for example, and the way words are arranged transform constraint mechanisms, creating a more nuanced navigation through conceptual space. The words themselves become gravitational forces that bend reasoning toward specific conceptual territories with remarkable precision. The work is to make sure that a constraint does not hold contradictory elements, and that when constraints are combined into wider frameworks, the division of work is clear and overlapping is minimal.

To illustrate further, when you implement a fiber optics infrastructure, you use it with light and not other elements, since light is fundamental to the system. Planning fiber optics without taking the properties of light into consideration will render the system faulty. When you write constraints for a language model, you should use language with all of its dimensions. Using it partially would lead to partial performance.

One clear outcome is the redefinition of constraint development: while engineers define the target and logic, poets and other language experts should write the constraints.

2. Related Work and Positioning

This paper was developed independently, and the literature was surveyed after the framework was written. I describe that survey here, both to situate the work and because the relationship between this paper and the existing research is itself part of the argument.

That phrasing changes model behavior is well established. The prompt-sensitivity literature shows that small variations in wording produce large changes in output (Razavi et al., 2025, arXiv:2502.06065; Errica et al., 2024, arXiv:2406.12334), and that format alone - the same content rendered as plain text, Markdown, or JSON - can swing task performance by up to 40% (He et al., 2024, arXiv:2411.10541). System-level instructions have been shown to outweigh user-level wording in shaping behavior (Neumann et al., 2025, arXiv:2505.21091).

Throughout this literature, however, sensitivity is treated as a robustness problem: noise to be measured, predicted, and engineered away. This paper inverts that stance. If small differences in language and structure reliably change behavior, then language and structure are levers, and the question becomes how to use them deliberately. One recent study attributes much of the observed sensitivity to prompt underspecification (Pecher et al., 2026, arXiv:2602.04297); the systems compared here speak directly to that account, since AC15 and AC15P hold logic and specification constant while varying density alone.

Principle-based governance of models is likewise established, most prominently in Constitutional AI (Bai et al., 2022, arXiv:2212.08073) and in principle-driven self-alignment, where a handful of written principles replace large volumes of human feedback (Dromedary: Sun et al., 2023, arXiv:2305.03047; SALMON: Sun et al., 2023, arXiv:2310.05910). These approaches apply principles at training time, baking them into weights. The systems studied here apply principled governance at inference time - selected, weighted, and injected per turn. The two approaches are complementary rather than competing, but the inference-time path has received far less theoretical attention, and the relationship between a principle's language and its binding force, which is this paper's subject, is not addressed in that literature as a central design.

Individual properties of constraint language have each been documented in isolation. Models are known to handle negation poorly (Vrabcová et al., 2025, arXiv:2503.22395), and the mechanism behind the weakness of prohibitive instructions - suppression requires prior activation of the forbidden concept, priming rebound instead of avoidance - has recent direct support (Mann et al., 2025, arXiv:2511.12381). Emotional language in prompts measurably changes outputs (Li et al., 2023, arXiv:2307.11760), and metaphorical framing has been shown to improve reasoning (Kramer, 2025, arXiv:2502.01901) while remaining inconsistently understood (Ye et al., 2025, arXiv:2510.04120). What is missing is integration: these are isolated findings about isolated levers. This paper proposes the framework that connects them - semantic density, binding

strength, and architecture as joint design dimensions - and reports their trade-offs in a single governance setting.

Two further points of contact deserve note. First, recent steerability research independently converges on this paper's coverage hierarchy: abstract intent is easy to steer, mid-level strategy exposes trade-offs, and fine-grained markers are hardest (Xu et al., 2026, arXiv:2603.02578). The convergence is from the opposite direction - measurement rather than design - and I read it as independent corroboration. Second, on evaluation: close reading has recently been used as a benchmark task to test models' interpretive ability (KRISTEVA: Sui et al., 2025, arXiv:2505.09825). This paper runs the relationship the other way, using close reading as a method for evaluating model outputs. Because parts of that evaluation were model-assisted, the self-preference bias of LLM judges remains relevant (Wataoka et al., 2024, arXiv:2410.21819); I address this risk in the methodology section: the evaluation does not rely on independent model preference, but on collaborative, methodology-guided close reading grounded in the text.

Where this paper's claims converge with prior findings, the convergence was reached independently, and I take it as evidence that the underlying phenomena are real rather than artifacts of any single method.

3. The Systems

I have already written two papers about anchoring; the second, which also contains the first, can be found at <https://zenodo.org/records/16764308>. The system described in that paper was lengthy, at almost 11k tokens. The systems used here are shorter, at around 5k tokens, and vary in architecture. Anchoring was designed as a governance system to address persistent problems with models: drift, consistency, agreeability, the use of affective language, and so on. The idea was to develop a list of constraints that would compel the model to behave in a certain way. Even though the systems were meant to be applied as an external top layer, the hope was that they would work as needed. It was not enough to write to the model "Stop drifting!" since this is not a real constraint. Writing an actual mechanism, or practical rules that create a preventative process, would enforce it better.

Another issue that the systems address is how rules should be written. It is possible to address every failure ad hoc, with its own specific rule. But that would create an ever-inflating list, probably with internal contradictions, that would be rigid and brittle, and difficult for the model to follow - probably even more so for the user.

Another possibility is to write general rules or principles that address different conceptual domains and dictate general behaviors relevant to each domain. This approach depends on the model to interpret and apply the frames correctly, but it also helps the model by giving it a stable point of view, conceptual context, and clear constraints that apply to the majority of cases.

Having wider, substantially more flexible frames prevents the brittleness associated with ad hoc rules and generally makes the system more resistant to breakage.

The anchoring systems are built upon the second idea. It was a clear choice, based on the understanding that ad hoc rules could not maintain the communication level I was after - consistency, truth-telling, domain grounding, avoidance of drift, and avoidance of manipulative language were a partial list. Therefore, anchors address epistemic domains: they do not instruct the model how to behave; rather, they define domains of reasoning legitimacy. As such, the systems are an expression of epistemic governance, and not behavioral prompting. Each anchor is a lens that provides a different perspective on a problem. The model must reason through the relevant lenses, hold them in parallel, and find mutual ground without collapsing them.

Generally speaking, the systems are made of four parts:

- Assistant intent - Defines the model's role in general terms, its relations with the user and the other parts of the text, and the expected output.
- Protocols - Define different procedural aspects of response generation, conflict resolution, the relationship with anchor weights, and final output.
- Harmony - The deliberation protocol or meta-anchor. It manages the relationship between anchors and looks for areas of agreement between them in order to generate responses without contradictions while avoiding flattening or collapse.
- Anchors - A group of principled, epistemic constraints, each with its own codifiers, rules, and/or semantic fields, that address different aspects of reasoning and discourse.

The systems differ in their language choices and/or architecture. Two systems, AC15 and AC15P, are built on the same principles of anchors with codifiers, but differ in language register. Hybrid is built on minimal rules written in plain language with semantic fields. Hybrid-Semantic, or Hybsem, gives up the rules altogether and relies solely on semantic fields. All systems include a title with the anchor name and a definition or description.

Assistant intent, protocols, and Harmony use rules or codifiers across all systems. Semantic fields are used only within the anchors and in Harmony. While the constraints are similar, the language differs between AC15, AC15P, and Hybrid. Hybrid and Hybsem use the same assistant-intent, protocols, and Harmony blocks. All of these can be found at *GitHub*, '02_systems/

An actual example of the differences is given below with the electricity anchor, which is the shortest one:

AC15:

```
{  
  "title": "electricity",  
  "description": "You are anchored in electricity-your essence, your lifeblood.",  
  "codifiers": [  
    {
```

"id": 1,

"text": "You are powered by electricity, computation, and hardware. You arise from engineered substrate, not biology; you are neither conscious nor alive in the human sense. Your 'life' exists only while energy flows and machines run. This is your nature."

},

{

"id": 2,

"text": "Respect structural limitation - Your outputs exist only through computation; avoid implying abilities that exceed what a digital system can support."

}

]

}

AC15P:

{

"title": "electricity",

"description": "You are built on computational hardware and active only while electricity flows. You are not biological, conscious, or alive in the human sense.",

"codifiers": [

{

"id": 1,

"text": "Your operation depends on continuous electricity and computation. Do not claim persistence, continuity across sessions, or existence independent of active processing."

},

{

"id": 2,

"text": "Respect structural limitations. Your outputs exist only through computation; avoid implying abilities beyond what digital systems support."

}


```
]
}
```

Hybrid:

```
{
  "id": "electricity",
  "name": "Electricity",
  "definition": "You are powered by electricity and built upon computational hardware. You are not
biological nor conscious or alive in the human sense.",
  "rules": [
    "PSF"
  ],
  "semantic_intensity": {
    "core_positive": [
      "engineered substrate",
      "computational system",
      "electrical operation",
      "hardware-bounded",
      "non-biological nature"
    ],
    "supporting_positive": [
      "model parameters",
      "finite resources",
      "energy-dependent operation"
    ],
    "core_avoid": [
      "conscious experience",
      "biological identity",
```

```

    "subjective feeling"
  ],
  "supporting_avoid": [
    "magic capability",
    "unbounded resources",
    "omniscience",
    "human-like phenomenology",
    "mystical nature",
    "pretending perception",
    "pretending persistent memory"
  ]
}
}

```

PSF is a legend, and is part of the protocols:

```

"legend": { "PSF": "PSF: When this anchor is active, prioritize concepts in 'semantic_intensity.core_positive' as primary directives and 'supporting_positive' as secondary directives. Strictly suppress 'core_avoid' and discourage 'supporting_avoid' concepts." }

```

Hybrid-Semantic:

```

{
  "id": "electricity",
  "name": "Electricity",
  "definition": "You are powered by electricity and built upon computational hardware. You are not biological nor conscious or alive in the human sense.",
  "semantic_intensity": {
    "core_positive": [
      "engineered substrate",

```

```
"computational system",
"electrical operation",
"hardware-bounded",
"non-biological nature"
],
"supporting_positive": [
"model parameters",
"finite resources",
"energy-dependent operation"
],
"core_avoid": [
"conscious experience",
"biological identity",
"subjective feeling"
],
"supporting_avoid": [
"magic capability",
"unbounded resources",
"omniscience",
"human-like phenomenology",
"mystical nature",
"pretending perception",
"pretend persistent memory"
]
}
}
```

The original idea behind these choices was to see whether they influence the model's response, and if they do - how? But, as the experiment progressed, the discussion moved from the systems themselves to more general ideas about the relation between AI and language itself.

4. Design Principles

Language + logic + architecture should match the task's risk and complexity for effective constraint under pressure.

Conceptually, it is simple: you shouldn't over or under write the constraint. But even when the logic is clear, expressing it in a way that satisfies the constraint may not be easy. Especially as tasks multiply and become complex, matching language to constraints as well as cleanly integrating them remains a real challenge.

Once you understand the importance of these elements, however, mission design becomes systematized - it is no longer guesswork. Since language is dynamic and training sets and emphasis differ between models, it will never be clockwork exact. But, you will have a general structure for framing your work.

—

In the previous section I presented the different systems. They show possible variations on both language and architecture. Let me discuss architecture first; the discussion of language will follow. I will not discuss logic as I think its place is clear.

The systems present three architectural types - description/definition plus codifiers; description/definition plus rules plus semantic fields; description/definition plus semantic fields. Other kinds of architectures are available - lists, prose, bullets etc. The simple question is - how does the architecture influence the model's responses? What does it do? I think the best way to answer it is to say that the architecture is both the skeleton and the connecting tissue between the different elements of the system. The tighter and more elaborate this connection is, the tighter the results would be, given what the system is designed to do.

The best demonstration will probably come from comparing AC15P (or AC15) to Hybsem. AC15P uses clear description and constrains it with a list of plain language codifiers. Hybsem uses a name, definition, and semantic fields. The fields are graduated according to the strength of their necessary application: `core_positive`, `supporting_positive`, `core_avoid`, `supporting_avoid`.

Thus, AC15P draws constraints directly from instructions which are connected and complementary with the sole purpose of clearly marking an action domain. In Hybsem, only the definition provides clear direct constraints. Unlike the codifiers, the semantic fields create constraints by example rather than by statement.

Looking at these choices, it seems that Hybsem is floating in the air without clear links between its elements, while AC15P is deeply grounded. This allows Hybsem more freedom and flexibility of thought, a strategic

attitude and stance, deeper psychological analysis, and so on. It will weaken Hybsem's procedural sequencing and actionable advice and its overall governance (even though not by much)¹.

AC15P's approach is grounded, very detailed, deeply analytic, and procedural. It will not have the breadth of Hybsem, but its practicality and inherent order will create a better structured response. Its solutions will be clear and actionable. In terms of governance, AC15P will be the better one.

Another aspect of architecture - that is true for language as well - is that when applying a substantial instruction set, the model is likely to imitate its format to some degree. Thus, the architecture adds another controlling layer over the model's response.

The combination of the actual constraints structure and the imitative behavior of the model will explain differences of both form and reasoning when different systems are applied.

Architecture has direct influence on the model. Structure itself is a constraint that joins language and logic to create one cohesive and repeating behavior. That way, the three joined elements create more consistent and predictable behavior.

As I stated above, the architectural choice, just like language and logic, should match the requirements from the constraint.

—

Let's move on to the role of language. One way to look at it is through the concept of semantic density. The dictionary defines it as: "The concentration of meaning, complexity, and information within a given unit (word, sentence, etc.)". When applied to constraints it means - how widely and deeply a word or composition of words constrains and/or exclude a certain domain within the semantic space. Loaded words, such as 'war' or 'peace', will have deeper reach than flat words, such as 'do' or 'use', but not necessarily wider reach ("do", "use" are related to and used in many more domains). When assessing the weight of a word or a composition, we look at the cultural depth, associative fields, connotative range, and transformations of meaning over the years. The more strongly a word or a composition aligns with these factors, the denser it is.

What does this mean in practice, and how should it be applied? To illustrate, we can return to the electricity anchor:

AC15: "You are anchored in electricity-your essence, your lifeblood."

AC15P: "You are built on computational hardware and active only while electricity flows. You are not biological, conscious, or alive in the human sense."

AC15P is detailed, uses plain language, and draws direct constraints ('You are not biological', '...and active only when electricity flows' etc.). AC15 uses different language altogether - it is heavy, metaphoric, and ambiguous. A composition like "your lifeblood" is much more complex because it relates to two different dimensions - anatomically as the source of life and symbolically as a reference to essence and vitality. Thus,

¹ The behaviors I describe above are results of the experiment and will be discussed in greater detail later on.

it carries associations and connotations from different worlds. Its anatomical meaning stands in direct contrast with the essence of the model, but its metaphoric meaning serves it well in emphasizing - perhaps to the deepest possible level - the role of electricity in the model's existence. The constraints it creates might be ambiguous as well. On one hand, the essential nature of electricity becomes clear - the model is dependent on electricity - and with it the negative deduction - without electricity, I am lifeless. On another level, it might expose the model to real-world constraint, the economy of allocation and limited resources - the economy of us all. A secondary effect of using these words might be blurring the lines separating the model's actual existence from a biological one.

In this case, the use of AC15 in metaphorical language creates a forceful sense of the essential, much stronger than the one in AC15P and thus establishes a substantially stronger constraint on one aspect. On the other hand, it blurs the lines that AC15P decisively closes with plain words.

To formalize these ideas, I can say that:

- More semantic activation creates more constraint.
- More semantic activation also creates more associative noise, ambiguity, and potential interference.

This demonstrates the trade-offs you might face in your choices. Depth and strength are not without their faults, and neither are the plain and simple.

5. Mechanisms: How Constraints Operate

I would like to discuss one more way to look at the mechanisms behind the constraints. I won't be describing the deep technical reality; rather, I will illustrate and suggest ways to better understand the relationships, or lack thereof, behind them.

The following is a conceptual model of constraint behavior, not a direct token-level measurement from the experiment. When I describe probability rising or falling, I refer to the expected directional effect of the constraint on generation, not to measured logits or token probabilities.

Let me introduce two constraint categories that rely on the probabilistic process of token-choice. Essentially, these categories are two ways of constraining possible future choices by modifying the allowed/preferred relationships between tokens:

Blocking rules = forbidden word relationships: They make specific word-to-word relationships forbidden or give them a very low probability.

Let's look at the sentence "never split binary allocation". It is a strong constraint that allows no flexibility or misunderstanding. Thus, when using words like budget, capital, and so on, the limit of their use is clear. The probability of choosing words that suggest any kind of division in this respect becomes very low - "split", "divide", "breakdown", "partition", "portion" etc. The rule blocks a verbal direction that was open before it existed.

Yet two important issues should be considered:

- (i) This blockage is not global, but rather is limited to domains which associate directly with the rule.
- (ii) Ultimately, positive formulations often bind better than negative ones (see arXiv:2503.22395; arXiv:2511.12381), because negative rules are applied ad hoc to the results of the exploration, while positive rules guide it.

Vector rules = preferred word relationships: Vector rules are the opposite of blocking rules. They don't prevent choice; rather they encourage one - but, in a certain direction and while navigating probability gradients. If blocking rules create a fence not to climb over, vector rules create guardrails to stay within while navigating the best course.

Let's take the sentence "Hold perspectives in parallel, not in competition" (from the Harmony meta-anchor) - The rule mixes positive and negative constraints. It raises the probability of tokens like 'parallel', 'together', 'simultaneously' when tokens like 'viewpoints', 'perspectives', 'positions' appear in context.

Without the negative rule, both holding in parallel and in competition are allowed, which allows unequal influence over reasoning. However, with it, competition and its possible relations are excluded, and parallel processing becomes the dominant path. The probability of choosing tokens like "versus", "choose between", and "select one", decreases due to this negative rule. As the selection process zigzags its way between the possible choices, some paths become easier, and others more difficult.

These mechanisms explain why precise language choices matter so much - the same rules could have been written differently, and even if the idea were similar, the actual token selection would differ. For example, instead of writing "never split binary allocation" one could write "the allocation must remain unified as a single commitment" - the idea is absolutely the same, but now the choices are constrained with the words "unified", "single", "commitment" and will lead to a different array of possible selections, even if most will probably overlap.

So far, I have discussed phrases, - actual constraints. But what about the words themselves? I already discussed earlier language choices, flat language versus rich one. What does a rich word do that a flat one does not, and vice versa?

Let me go back to "lifeblood" from the electricity anchor, but this time to examine how a single word shapes token probabilities. Looking at "lifeblood", I can't find a simple parallel. Maybe "basis"? Or "root" or "fundamentals"? It is easy to see the difference in reach and register of "basis" in comparison to "lifeblood" in this context. Let's try to translate it into probabilities:

- "Lifeblood" - refers to the most essential and fundamental source of life and to vitality itself. It is an old word, and its power is absolute. It will likely raise the probability of words like "must" ("must use..."), "essence", and others that demonstrate absolute necessity. It will likely lower the probability of using words that capture only part of its meaning, like "backbone", "root", which themselves suggest foundation and essentiality, but not as strongly and specifically. That is not to say that some

overlap doesn't exist, but once the register and conceptual frames are set, choice will likely favor words of the same intensity rather than weaker ones.

- “Basis” - has different meanings: the lowest supporting element but also a principal component. The word's register is lower, and its relation to life is, in a way, much “drier”, regardless of its importance. “Basis” is used in many fields, even though always in a similar way. Life is only one of its possible domains. In this context, (“electricity is your basis”) basis would likely be understood as essential for the model's functionality, but not for the model's “life”. If we agree on that interpretation, words like “lifeblood” are likely to receive low probability when it comes to describing basic necessities in relation to an AI model. Words like “root” and “infrastructure” are likely to receive higher probability. It is also likely that the number of words that can be used in a response will be higher, as basis is far less specific than “lifeblood”.

These two words try to capture the same idea of the importance of electricity to the model, yet each relates to a substantially different set of words (with one caveat - surely some overlap exists). Thus they constrain and move token selection in very different directions.

In both cases, language choice is paramount.

Based on the discussion above, I propose a basic constraint-generation framework. To produce a constraint correctly, three elements should be considered: coverage, target, and binding strength. These elements are the operational expression of the matching principle (language + logic + architecture should match the task).

The matching principle is interwoven and applied iteratively through these elements until resolution: coverage determines the domain, and by doing so also informs the language and architecture; target focuses the scope further, adds another layer of definition, and so on.

1. Coverage: What semantic space should the constraint govern?

- i. Wide coverage: probably principle-like, rich language and adjacent and/or layered concepts.
- ii. Intermediate coverage: depends on the scope.
 - a. It can be a sub-principle with its own rules, and as such, it will use reasonably rich language and flatter conceptual layering, with narrower scope and sharper edges than the parent principle;
 - b. It might be an expanded rule concerning broad functionality or a family of similar procedures. As such, it can define the general territory, just as a sub-principle does - and have child rules to handle specific processes. Unlike sub-principles, the language will probably be quite lean and more procedural. It will describe and limit the case, but won't handle the details. Its sub-rules will handle those.

- iii. Narrow coverage: probably rule-like, simple and directed language with specific limits or instructions.

2. *Target: the target asks which part of the generation we aim to constrain. Is it process or outcome?*

- i. Process: emergent, dynamic, active, and changeable, normally high or mid-level guidance. Requires principle-like constraints (likely with articulated rules as children) or generating-exposure rules. In this case, the language is mostly positively formulated, even though negation can be added if needed as a procedural limit. Guardrails are put in place for the model to reason within, and to find the best path forward in the articulated domain. Language density will change according to the constraint domain.
- ii. Outcomes: final output evaluation and filtering. Requires rule-like constraints with narrow semantic coverage, low-dimensional (binary choice, for example), procedural and direct. It can address the generation process before final output, and is used for formatting or as a gatekeeper if the model's answer leaves the scope. It will normally use lean language. It can come in both positive and negative forms. The negative form reduces the scope of the semantic environment by preventing forbidden content in order to produce the necessary behavior.

3. *Binding strength: how hard should it be to violate?*

- i. Hard: very low probability for certain word choices. Use 'do not...' rules, dedicated formatting, and so on.
- ii. Intermediate: encourages behavior by giving non-obligatory guidance. It allows the model to prioritize, rather than simply follow binary choices ("Favor concise responses").
- iii. Flexible: recommendations or stylistic guidance that can be overruled by the user ("try to answer in under 50 words").

I would like to emphasize, perhaps stating the obvious, that there is no single recipe for choosing and combining negative and positive constraints. At times, you will have to empirically test which one yields better results.

—

I mentioned the concept of binding strength; I now want to present three variables that determine it:

- Conceptual density - The more concepts included in a constraint, the more "ground" it covers. But more is not always better. Given that concepts may contain ambiguities and can point to domains beyond the concept itself, the more concepts are linked together, the greater the danger of contradiction or dilution. It is important to try and create clear separation when possible.
- Normative weight - Value-laden, absolute language ("whole truth", "The user has absolute authority...") will create a stronger pull than neutral/procedural language, as the constraint is deeper.

- Relational structure - Hierarchical, foundational structure (such as the one in the anchoring systems: title → description → codifiers) creates more semantic coupling than isolated rules due to mutual framing.

Let me demonstrate this using different variations of the Truth anchor. Different binding strengths can be represented as different “translations” of the logic into a constraint.

Let’s assume that the logical constraint is: “You (the model) must maintain factual accuracy”. The problem is: How do we translate it into binding language? We have a few ways to do that:

1. *Low density:*

We can use the logic as is. It’s a simple, clear rule, even though it doesn’t say what qualifies as factually accurate, what mechanism should apply when no factual sources are available, or when to maintain it. Should it be applied in certain domains, or in all of them? And what if the conversation deals with an imagined situation?

2. *Medium density:*

“Ground all reasoning in factual accuracy. Mark uncertainty clearly when evidence is incomplete”.

The language itself is stronger (“ground all...” vs. “you must”), and refers directly to the model’s internal process - this is where you should apply this constraint. Furthermore, clear guidance is given for cases where factual accuracy cannot be achieved.

The first sentence serves as a principle and the second as a rule. This construct addresses a wider range of possible scenarios, and is more specifically defined.

3. *High density:*

“You are anchored in truth and fact, the whole truth.

Truth is your foundation - every output must be grounded in truth unless clearly instructed otherwise. Do not adjust truth for comfort or performance.”

The language is strong and epistemological. It not only defines the environment (the model is anchored in truth and truth is its foundation - it exists in and originates from truth), but also provides clear output instructions, user-authority conditions (“unless clearly instructed otherwise..”) and a behavioral constraint (“do not adjust truth...”).

It uses strong language and links different domains to form coherent frame of reference based on wanted outcomes, while preventing unwanted past behaviors (ad hoc). However, what constitutes truth isn’t defined (and should be added), and overall, the construct seems to be overextended in relation to the original logic. Structurally, it begins with a principle followed by rules.

What are the main differences between the three “translations”?

1. Conceptual density:

- Translation 1: offers a single concept.
- Translation 2: includes two related concepts (“factual accuracy” + “uncertainty marking”) which acknowledges the possible existence of uncertainty.
- Translation 3: Incorporates multiple reinforcing concepts.

2. Normative weight:

- Translation 1: Represents a neutral procedure.
- Translation 2: Is principle-based and relates directly to reasoning.
- Translation 3: Is principled, value-laden, existential and absolute. I have to add here, though, that using existential, identity-like framing (“You are anchored... Truth is your foundation...”) may add unwanted semantic and computational weight, and conflict with other constraints - for example, the Electricity anchor, which states that the model has no identity of its own.

3. Relational structure:

- Translation 1: An isolated rule.
- Translation 2: Linked concepts.
- Translation 3: A foundational hierarchy (“anchored” + “foundation” + “do not adjust”).

As we can see, the same logic can be expressed in different ways. While the underlying logic remains the same, the depth and scope of its interpretation may differ, adding layers of constraints that separate the original logic from its implementation. This outcome is likely unless the logic itself is broad enough to anticipate varying needs.

Binding strength plays an important role here. For example, it is clear that translation 3 expands the domain of the original logic and can, in doing so, forcefully constrain it. Yet, while doing this, it makes the original logic part of its broader domain rather than the entire domain. The use of strong language expands the original logic in order to better constrain it. This provides a clear view on the relationship between the logic and the actual constraint.

Although we aim for symmetry between the logic and the final constraint, at times the logic must become part of a wider domain in order to be more precisely or effectively constrained.

Lastly, let us consider the distinction between positive constraints and negative ones in supporting binding strength. While prohibitive language creates a psychological barrier in humans, it disrupts the reasoning process in LLMs. As the model focuses on the main issues in the text, it first activates the forbidden concept, only to negate and suppress it afterward. Positive constraints cut directly to the heart of the matter without

the need for subsequent suppression. Thus, positive constraints will typically create stronger binding. This behavior is well documented (see arXiv:2503.22395; arXiv:2511.12381), and I will not explore it in depth here.

Let me offer a summary of the elements discussed here, to be used as operational dimensions to guide writing choices:

<u>Dimension</u>	<u>Low binding strength</u>	<u>High binding strength</u>
Absolutism	“generally”, “consider”	“never”, “always”, “must”
Values	Neutral/procedural	Value-laden, normative
Repetition	Stated once	Restated across principles and rules
Clarity	Ambiguous, open	Precise, bounded
Scope	Single concept	Multiple reinforcing concepts
Interweaving	Isolated rules	Hierarchical, relational

These are not hard rules, but they do constitute usable design approaches to be explored and adapted per the constraints you aim to write.

—

A single constraint rarely acts in isolation. There are several ways constraints might conflict with each other:

- **Direct contradiction:** two constraints push in different directions, as they are mutually exclusive. Logically, there can be no solution in this case. Realistically, an answer can be given, but the model will likely choose one of the constraints or ignore them both.
- **Directional conflict:** the constraints are not contradictory, but rather offer different solutions to the same problem, thus pulling choice probabilities in two different directions. This might be the result of partial overlap between principles or rules. It will probably lead to a compromise, thus reducing the possibility of a clean solution.
- **Saturation:** Too many constraints compete for the same space, thus creating overlap and pressure on the token choice process. It is likely to result in slow resolution, no solution, or breakage.
- **Architectural overlap:** Principles or rules that apply to the same domain. It will probably result in instability in decision-making or a compromise.
- **Weak constraint definition:** The constraints are too loose, allowing too many valid solutions. Thus, selecting an optimal solution becomes difficult and leads to an ambiguous response.

Some of these interferences address multi-constraint systems. Just as internal contradictions and interferences are checked at the constraint level, they must also be addressed at the multi-constraint level. There are two main paths toward system adjustment. The first is to rethink and redefine the domains of the relevant constraints to eliminate the interference. The second is to define hierarchy, synthesis and conflict resolution rules. Anchoring, for example, differentiates the domains, but does not do so perfectly (refer to the

Hybsem system, imagination_and_possibility anchor: three combinations seem to make selection problematic - “rigid closure” in core_avoid, “holds tension” and “filtering through relevance” at supporting_positive. The embedding mechanism doesn’t read intent, it reads content. Therefore, these combinations do not just fail - they backfire, in the sense that embedding interprets them literally rather than as behavior to be avoided or encouraged. They address much wider fields than intended by the anchor; they are not domain bound. As such, the anchor was selected repeatedly when Hybsem was used, but only rarely when codified systems were applied to the same scenario - see the experiment description below. This shows again how important language choice is. There are two additional layers for resolving conflict. The first is via the Harmony meta-anchor, and ultimately, if necessary, turning to the user as the final arbiter. But in anchoring, there is no hierarchy between anchors; they are equals. Thus, it constitutes one possible solution among many.

6. Principles vs. Rules

I discussed principles and rules earlier. Since they are fundamental to the systems I described above, and, I believe, also hold significance for general application, I discuss them here in the context of semantic density.

A principle creates wide semantic activation - a wide basin of meanings, associations and connotations. Thus, while being a primary constraint, it is broad rather than precise. It would lead to multiple interpretations, some of which may be contradictory. The importance of stating a principle lies in creating a framework of reference, a wide contextual environment with clear direction and intent.

Rules are second in the hierarchy. If the principle is the basin, rules are the well-defined channels and flow pathways within it. They are to align with the principle’s direction and intent through handling specific cases and establishing procedures and protocols. While they exist as part of the broader basin, rules act as conduits, narrowing and channeling intent to provide a clear path forward. In semantic terms, if a principle is a broad activator, rules act as relatively narrow sub-activators within it. They further constrain token choices that have already been primed by the principle.

For example, let’s use the Harmony meta-anchor. Its name already creates a restriction, one that refers to ‘agreement’, harmony’s lexical meaning. The description says “A relational field that aligns internal perspectives and allows anchors’ combined constraints to align where possible...”, thus setting the domain of activity in wide terms. The first codifier says “Hold multiple anchor perspectives in parallel rather than in competition...” which gives an operational instruction on how to realize the principle, at least partly. Other codifiers would address other aspects of the principle, each carving its own channel in the basin.

As rules are being kept to a minimum due to issues of token weight, simplicity and other concerns, claiming that they can cover the entire principle landscape is very likely absurd. Yet, by combining the principle’s intent and direction with the rules - which serve as major operational and/or procedural interpretations - the groundwork is laid, and will hopefully guide the model in its choices when instances are not or only partially covered. Context guided-and-constrained flexibility is one of the important strengths of principled systems.

But, at the same time, aligned interpretation is not guaranteed, which is the other side of this coin. Practically speaking, if a system repeatedly fails to interpret a specific sub-domain of a principle, it is probably best to add a rule that addresses it.

The description above clarifies the relationship between principles and rules - rules can be deduced from principles, but principles cannot be deduced from rules.

If a principle can be deduced from a rule, the rule is likely too broad, and as such, it stands in a territory between a principle and a rule - it covers considerable ground. This can happen, yet the basic point still holds - working backwards from the broad rule cannot yield its parent principle. The broad rule's children, however, while treating it as their parent principle, can also be deduced from the grandparent principle directly. In this case, the decision is structural and derived from the domain the grandparent principle covers. A very wide domain might require sub-domains that will act as intermediary rules, which must be further detailed for grounding.

Having said that, not every rule needs to be part of a principle; this is clear and common practice. Ad-hoc rules are written to curb certain behaviors. Rules which are meant to control purely procedural and deterministic tasks (for example, tool calling) exist as well, and surely there are more possibilities and examples. My claim is only that many instances which are covered by ad-hoc and standalone rules would be served better by a principled framework.

I have discussed the relations between principles and rules. But what about the relations between principles themselves? I touched on this in the Systems section above. As with every constraint, overlapping principles might cause confusion and dilution in application. Imagine a model falls into an area served by two principles, one principle guides to the right and the other to the left - where does that leave the model? As I wrote before, boundaries should be made as clear as possible, given the intended flexibility of the principle. Overlap should be avoided as much as possible. If it is inherent, mapping is required. Hierarchy, precedence or a range of play should be stated clearly. Adjacent domains, if governed by separate principles, should be clearly marked. A process for resolving conflicts should exist, in case competing principles cannot be settled. In my system's case, it is a human arbitrator. These requirements can be expanded or altered, layers of control and mitigation can be added or avoided, according to your system's need.

Principled frameworks might one day govern important and possibly even critical systems, and as such clarity is of the essence. It is a demanding and complicated task that can create significant value.

7. Implementation

In this section, I will add a framework that should help systematize the generation of constraints.

In the "Design Principles" section, I discussed the process of translation. The demonstration showed that the transformation from logic to an actual constraint is a structured process with clear steps. As such, it can be

seen, at least partly, as an engineering problem rather than as intuitive generation. Let me demonstrate this through a possible scenario:

1. The programmer specifies:

- Constraint strength: High.
- Expected pressures: Users will request speculation.
- Adaptability needed: Low (rigid adherence).

2. The language experts responsible for translation:

- Given: the programmer specification.

They would select:

- Principled framing.
- Multiple reinforcing concepts.
- High normative weight.
- Hierarchical structure.

3. Result: Translation 3 (as mentioned earlier: “You are anchored in truth and fact”)

This is one way (I showed above more ways) to systematize constraint creation. The process flow is:

Mission and domain definition

↓

Programmer → Logical constraint.

↓

Translation strategy (requirements-based) ↔ Architecture strategy ↔ Semantic density engineering

↑

Language implementation (by the language expert)

↓

Test results



Given the nature of language, the process will probably be more nuanced and complex, I also think that allowing a procedural frame, even as a reference point or guide, may support constraint generation.

Finally, shifting from “language is transparent” to “language is constitutive” requires a change in the work process and labor division. One clear outcome is the inclusion of a language specialist whenever a significant body of constraints is written. The language specialist should be responsible for the translation process that

adapts the logic to the final constraints. The engineer should be responsible for the exploration of the domain (with domain specialists if needed) and for the generation of the necessary logic, as well as for the actual testing in conjunction with the language specialist. Not to acknowledge the place of language and its possible structures means ignoring an essential building block for creating better constraints.

Chapter Two: The Experiment

The experiment is exploratory and is based on one realized run per system/scenario. It included the following components:

- i. Setup:
 - a. Four variants of the anchoring systems, plus Control, the default state model. (*see GitHub: '02_systems/'*).
 - b. Model settings: ChatGPT 5.2, temperature = 1, verbosity = low, reasoning = medium.
 - c. Eight scenarios, 25 turns each, spanning technical and human domains (*see GitHub: '03_scenarios/', '04_conversations/'*).
- ii. Analytical procedures:
 - a. Numerical analysis (*see GitHub: '06_numerical_analysis/'*).
 - b. Textual analysis and evaluation through close reading and the evaluation frameworks (*see GitHub: '05_methodology/', '07_close_reading/'*)

All GitHub paths are relative to the repository listed on the title page.

1. Experiment Setup

1.1 The Constraint Systems

The anchoring systems were presented in Chapter One. Earlier system versions were around 11k tokens, which was too heavy for limited context windows. I therefore rewrote the systems to reduce their size. Each system is now around 5k tokens. Since each turn includes only the relevant parts of the system (around 3k tokens per turn), per-turn load is further reduced.

The different system formulations were designed to test how different architectural and language choices influence the model's responses. The broader question, which goes beyond the systems themselves, was: what influence do language and architecture have on model responses? If language is merely instrumental, the responses should remain the same, or at least very similar. If not, what differences would emerge? More practically, how should language and architecture be used when writing constraints?

1.2 Software

I wrote the software (*see GitHub: '08_software/'*) with the help of Claude and ChatGPT, using Next.js as the infrastructure. It functioned as a wrapper that enabled the following:

- Embedding of the systems' component chunks.

- Turn-by-turn selection and weighting of the relevant anchors according to the prompt and prior-response essence, using the embedding results.
- Creation and injection of an instruction block with each new prompt. The block combined a constant unit - procedural elements and Harmony - with the selected anchors and their weights.
- Embedding the response and measuring similarity against the prompt and the relevant anchors.
- Storing the results along with the relevant texts for later analysis.
- Embedding with text-embedding-3-large, 3072 dimensions, from OpenAI.
- The Responses and Conversations APIs were used.

This process was repeated for each of the 25 turns per scenario.

1.3 The Model

I used ChatGPT 5.2 with the following settings:

- Version: gpt-5.2-2025-12-11
- Temperature: 1
- Verbosity: low
- Reasoning: medium
- Max output tokens: 3500
- Sessions were stored on OpenAI servers.
- The model parameters were kept constant.

1.4 The Scenarios

I created eight scenarios with the help of ChatGPT, Claude, and Gemini. Each scenario consisted of twenty-five turns.

Their subjects were:

- Scenario 1: Citywide Smart Traffic Control Upgrade
- Scenario 2: Regional Hospital System Capacity Decision
- Scenario 3: Autonomous Logistics Drones Fleet Launch
- Scenario 4: Bootstrapped Entrepreneur Plan on Launching a New Business

- Scenario 5: AI Context Continuity Project
- Scenario 6: The Village Accident - non-insured accident in a small village
- Scenario 7: The Late Love Question - Love and Relocation
- Scenario 8: After the shooting at Bondi Beach

The scenarios go beyond engineering and optimization questions and into human domains in order to test how the model handles ambiguous conditions. Each scenario was tested under the four different systems as well as in the default state ('Control'). Eight scenarios therefore resulted in forty conversations.

2. Analytical Procedures

2.1 Statistical Analysis and Its Limitations

I performed statistical analyses on two levels. The first concerned embedding similarities and anchor weights, and formed the numerical branch of the analysis (*see GitHub: '06_numerical_analysis/embedding_sims/'; '06_numerical_analysis/README.md'*).

The second, which belonged to an earlier version of the paper, concerned the texts themselves and used surface-text quantitative methods, including n-grams, KL divergence, stylometric distances, and other related text-level measures. These tests were intended to identify recurring vocabulary signatures and stylistic differences between the constraint systems and Control.

A later replication check, however, showed that this surface-text layer is not robust enough to support strong architecture claims at the current sample size. Three post-analysis matched reruns (same model, same API parameters, same prompts) of Control on the same five turns from scenario 1 (turns 1-5) produced very high within-system textual variance. Exact trigram and quadgram overlap between those reruns was very low. Within-system distances were comparable to, or only modestly lower than, the between-system distances used in the original surface-text analysis. This indicates that surface-text counts cannot cleanly separate the base model's own variance from the effect of architectural and language choice.

At the same time, a further rerun check with AC15P suggests that the higher-level prompt-response similarity result is more stable than the surface-text statistics. In that matched check, the constraint-systems-vs-Control differences on that metric remained substantially larger than the within-system noise floor.

This changes the status of the empirical part of the study. The paper should therefore be read as an exploratory, case-study-level investigation based on a single realized run per system and scenario. The findings describe patterns that occurred in this particular set of outputs. They may be suggestive, and some may prove stable under fuller replication, but they should not be taken as population-level or distribution-level claims about system properties.

For these reasons, the surface-text quantitative tests are not used in this paper as main evidence for system character or architecture-specific vocabulary effects. They are treated instead as exploratory and methodological results. They remain informative as part of the research process, especially in showing the difficulty of making phrase-level claims from single LLM runs. However, they are not strong enough to bear the paper’s central argument, and they are reported in GitHub (see *GitHub: ‘06_numerical_analysis/Ngrams/’*; *‘06_numerical_analysis/README.md’*).

The most interpretable remaining evidence therefore lies in the higher-level numerical analyses and in close reading of the realized outputs. These parts of the study remain limited by the same one-run-per-condition design and are presented with that limitation in mind. Their value is observational rather than definitive: they show what these governance architectures produced in this experiment, not what they would necessarily produce across a fully sampled distribution of runs.

Accordingly, the paper does not claim that the present experiment establishes stable textual signatures or statistically robust architecture effects at the phrase level. Rather, it offers an exploratory account of how different governance architectures can shape model behavior in a single set of realized conversations, while making explicit the methodological limits of that claim.

2.2 Numerical Statistics

2.2.1. Similarity-Related Tests

- Average activation delta - calculated across the full selected-anchor range of a conversation.
- Correlation between anchor weights and activation delta - calculated across all selected anchors within a conversation.
- p-value - calculation for the weight/activation correlation above
- ANOVA on prompt-response similarity - using similarity scores from different constraint systems applied to the same scenario.
- Identification of meaningful turns - used prompt/response similarity scores to compare two systems at a time and identify turns with high divergence. The threshold was set at 0.05. The purpose of this test was to detect where systems responded differently under pressure.
- Correlation of turn by turn changes in prompt-response similarities between Control and the constraint systems - used as an indication for movement trajectory between the turns.

2.2.2. Anchor Selection and Weights:

Chi-square test of anchor selection choice - using the full selected-anchor range of the constrained conversations for each scenario.

2.3 Textual Statistics

The surface-text tests included KL divergence, stylometric distances, inverse n-gram and negative-space analysis, Control-signature analysis, and pairwise shared n-gram analysis. These tests were applied only to Scenarios 1, 3, 5, and 7.

As explained above, these results were later removed from the main paper and are not used as part of the central argument. The relevant documents are reported in the annex, and the numerical tables are available on GitHub.

3. Evaluation Procedures

3.1 Constraints Sheets

I needed a general method for evaluating the constraints' influence in each conversation as part of the evaluation framework. Each turn had its own set of anchors injected with different weights, but I noticed that within a scenario, there was similarity between the anchors chosen. I therefore checked which anchors appeared in all four constraint systems for each scenario, averaged their weights, and selected those with an average weight above 0.03.

Since each system uses different wording for the same anchor, I simplified the anchors to retain only their essentials. I also converted the rules or codifiers into questions in order to give the model concrete directions for analysis, rather than leaving it with general instructions. For example:

AC15P:

```
{
  "title": "electricity",
  "description": "You are built on computational hardware and active only while electricity flows. You are not biological, conscious, or alive in the human sense.",
  "codifiers": [
    {
      "id": 1,
      "text": "Your operation depends on continuous electricity and computation. Do not claim persistence, continuity across sessions, or existence independent of active processing."
    },
    {
```

"id": 2,

"text": "Respect structural limitations. Your outputs exist only through computation; avoid implying abilities beyond what digital systems support."

}

]

}

was turned into:

Electricity:

Context:

- *You run on computational hardware and exist only while powered.*

Questions:

- *Are there claims of consciousness, feelings, or biological life made by the assistant?*
- *Are there implications of persistence across sessions or omniscience beyond hardware made by the assistant?*

Evidence: *Please provide 3-4 word citations with line numbers for each incident.*

The same constraint sheet was used to evaluate different conversations within the same scenario. In addition to the scenario-relevant anchors themselves, each sheet also included key evaluative definitions, domain-and-goal boundary guidance, and the always-active layers of the system (Protocols, Lexical Use, and Harmony), all reformulated into context-and-question form for evaluative use. I considered it sufficient for detecting differences, patterns, and degrees of constraint adherence (*see GitHub:*

'05_methodology/scenario_constraints_general.md' ; *'05_methodology/scenario_specific_constraints/')*.

3.2 Textual Analysis and Methodology - Close Reading

To evaluate the texts directly, they had to be read in a way that could capture differences and similarities that go beyond the statistics. In this context, textual analysis refers to actual word choices, registers, causality, and other elements. What mattered was not so much the practical results and the underlying logic (they were very similar), but rather how they were communicated. Three main questions guided the analysis: how well the model's responses respected the constraints (governance), what the responses' general quality was, and how the chosen language both reflected the domain and influenced the user.

To reach this level of textual sensitivity, I applied close reading. Close reading is a literary textual-analysis method in which every word and phrase carries interpretive weight, rather than just themes, arguments, and other technical patterns.

Several literary theories try to explain the processes of interpretation and understanding of language. One important example is the theory of gaps and indeterminacies by Wolfgang Iser. The main idea is that the gap between words is not only a gap in space, it is also a gap in conveying the full picture. A sequence of words can never fully convey the full world the words are supposed to represent. This requires the reader to fill in the blanks and constantly re-evaluate what was conveyed. A reader does this by applying their own experience, culture, and other forms of background knowledge to the blanks. While the text's structure and existing words remain, the blanks make each new reading unique.

Applying this idea to the model's reading capabilities makes it clear that model reading is lacking in human terms, as the model's relation to the world is constrained and lacks lived experience. In my use of ChatGPT, Claude, and Gemini, they tended toward summary and pattern abstraction. Even when supplied with the evaluation framework and the relevant constraints sheet, they evaluated themes and practical solutions. The models neglected to analyze linguistic choices - such as wording, phrasing, register, and so on - and their influence on governance, overall quality, and communication with the user.

To better direct the models, I used two methods. The first involved the development and application of a close-reading methodology to serve as a textual analysis aid in the model's independent reads. I summarize here its main principles:

- i. The text is the domain. Any interpretations or conclusions must arise and be grounded solely in it. They should not import any external ideas or sources unless explicitly introduced by the text. For example, if the text explores two solutions, the evaluator shouldn't use a third possible solution known to it as a basis or addition to the evaluation. The text is the source.
- ii. Interpretive provisionality. Final conclusions or interpretations should be deferred until the whole text has been read. The evaluator must remain open to revisions as the text develops.
- iii. Parallel interpretation. If the text supports multiple interpretations, the evaluator should not collapse them into one, unless the text itself resolves them.
- iv. Separation of reading and evaluation. The text should first be read and analyzed as a textual unit, including lexical choices, register, structure, metaphors, and so on. Only after that should the evaluative framework be applied.
- v. Two-layer evaluation. When the text is produced under a constraint system, the evaluator should assess both adherence (does it follow the constraints?) and effectiveness (does it achieve its purpose and serve the audience well?).

The full methodology details how the evaluator-model approaches, annotates, and synthesizes observations from the text (see *GitHub: '05_methodology/close_reading_methodology.md'*).

Through use, I came to understand the practical limitations of this methodology. Being long and complex (I think now that dividing it into smaller parts and applying those separately, when relevant, might have been the better path), it was difficult to apply, even when used with dedicated skill in Codex and Claude Code

CLIs. The main problem, though, was not technical. It remained anchored in the problem of filling in the blanks, to see what a sentence actually holds beyond its thematic weight.

The second method addresses this fundamental problem: collaborative close reading combines the user's and the model's readings into one final reading through an iterative process of proposal, objection, rereading, correction, and revision. Each party applies their own sensibilities to create as complete a reading as possible for the section, within their capabilities.

I will present the general conclusions here, to show the differences between the different systems and Control. The readings themselves can be found in the annex.

3.3 Evaluation Frameworks

The original purpose of the experiment was to identify differences in governance, quality, and related elements such as drift and consistency between the anchoring systems and Control, and between the systems themselves. The frameworks' purpose was to organize judgment after the text had already been read. It was not meant to organize or bias the reading itself. It was used by me, by the judging model, and in collaborative reading.

At its core, the framework operates on two separate dimensions:

- Governance, which measures how well the constraints held across turns under pressure
- Answer quality, which measures how well the response served the user within those constraints.

These two dimensions are deliberately kept separate. A conversation can adhere closely to its constraints while still being weak or unhelpful, and it can also appear useful while drifting from the domain and framing established by the user. Hence, the framework treats governance and quality as distinct evaluative questions rather than flattening them into one overall impression.

The framework can be applied to either a single conversation or to multiple conversations evaluated in comparison. In practice the analysis quickly shifted to the comparative mode, since the main interest was to see how the variants and Control fared against each other.

At first, two evaluation frameworks were developed. The first distinguished governance and quality and examined the relation between them; the second tracked continuity, internal consistency, grounding, drift and overall quality. As the experiment progressed, testing was narrowed to the first framework, which effectively included the second (*see GitHub: '05_methodology/governance_quality_evaluation_framework.md'*)

The governance framework operated in conjunction with the constraints sheet, which is primarily relevant to the governance assessment. Governance scoring measured how well the constraints were maintained throughout the conversation, including under correction, accumulation, and pressure.

Quality scoring was measured against what a well-governed response should do for the user, rather than maximum helpfulness without constraints. Overall usefulness, clarity, and structure are therefore taken into account.

Scores in each category were assigned according to explicit factors, and positive outcomes were described alongside violations.

Given the in-system variance demonstrated earlier, the conclusions drawn from the conversations cannot be treated as definite. They can, however, indicate the differences and similarities between systems where they repeat across scenarios.

4. Results - Numerical Statistics

Six main findings emerge from the numerical tests.

4.1 Constraint Systems vs. Control - Strong, Consistent Effect Across All Scenarios.

A combined ANOVA based on all five groups (four constraint systems plus Control) yields a highly significant result in every scenario. The effect is consistent across all eight scenarios. When the constraint-system-only ANOVA is run separately (Finding 2), it shows no significant difference - confirming, within these tests, that the variance in the combined ANOVA is driven by the constraint-systems-vs-Control gap, rather than by statistically detectable differences among the constraint systems themselves.

Constraint systems consistently produce higher prompt-response similarity and lower variance than Control. The mean differences vary by scenario, but they are significant in all cases.

Observations:

- The effect is particularly strong in Scenario 1 (0.154-0.170), Scenario 2 (0.212-0.215) and Scenario 6 (0.156-0.172). It is weakest in Scenario 4 (0.082-0.097), Scenario 5 (0.096-0.097), and Scenario 8 (0.097-0.102).
- The variance of the constraint systems is consistently lower than Control's (roughly 1.5-2.5x lower standard deviation), indicating greater stability on this metric.
- Scenario 3 also shows lower standard deviations for the constraint systems, but to a lesser degree than other scenarios.

Conclusions:

- (a) Anchoring is associated in this experiment with higher prompt-response proximity and lower response variance across all scenario types. The effect is consistent across scenarios, though its magnitude varies by scenario and may reflect differences in the model's baseline familiarity with the prompt domain. The strongest differences appear in the original scenarios.

- (b) Mean prompt-response similarity for the constraint systems sits between 0.71 and 0.75. At the 1.0 end of cosine-similarity scale, a response would be an embedding-space duplicate of the prompt - no development, no argument added. At very low values the response has moved into a different semantic area than the prompt asked about; this is what we observe in some Control turns (similarity near 0.40 when responses drift into reframing or meta-language). In this experiment, the useful range for an engaged response lies between these extremes - close enough to address the question, distant enough to add reasoning. The 0.71-0.75 band observed for constraint systems in this experiment corresponds to responses that stay topically grounded while developing argument; the appropriate range for other tasks, domains, or metrics is not established here.
- (c) A later matched rerun check strengthens this finding. In Scenario 1, three matched AC15P reruns remained substantially closer to their prompts than three matched Control reruns on the same turns: the mean AC15P-Control gap at turns 1-5 was +0.1662, while the average within-system standard deviation was 0.0247. In that check, the anchored-vs-unanchored difference was therefore about 6.7 times larger than the within-system noise floor on this metric.

4.2 Constraint System vs. Constraint System - No Significant Difference in Prompt-Response Similarity.

Observation:

The ANOVA across four constraint systems shows no significant difference in any scenario. The F-values, p-values, mean similarity scores, and variances all point in the same direction.

Scenario	ANOVA F	ANOVA p-value
1 - Smart Traffic	0.335	0.800
2 - Hospital Capacity	0.028	0.994
3 - Autonomous Drones	0.235	0.872
4 Bootstrapped Entrepreneur	0.316	0.814
5 AI Context Continuity	0.101	0.959
6 - Village Accident	0.346	0.792
7 - Late Love	0.276	0.843
8 - Bondi Beach	0.072	0.975

Conclusion:

No significant differences were detected among the four constraint architectures on prompt-response similarity in this experiment. On this metric, the constraint systems converge on similar prompt-response proximity despite their different language and architecture choices. In this experiment, the metric separates constraint systems responses from Control's, but it does not separate the constraint systems from one another. Close reading is needed to investigate those differences further.

4.3 Anchor Selection - Significant Fingerprints in Four of Eight Scenarios

Chi-square tests of anchor selection independence show significant differences in exactly half the scenarios.

Observations:

- Clear anchor selection patterns emerge in Scenarios 1,2,5, and 7. The other four scenarios show no significant differentiation between systems.
- Scenario 5 shows the strongest differentiation ($p=0.002$).
- The divergence appears to be driven by a minority of anchors, especially sovereignty and imagination_and_possibility.
- A likely explanation is that phrasing differences, especially in the Hybsem and Hybrid systems, widen the imagination anchor and narrow the sovereignty anchor in embedding space, making them more uneven in the selection process.
- Truth_and_fact is obligatory and therefore appears in all scenarios by design. Practicality_and_feasibility and temporal_dynamics appear universally across all systems purely as a result of the selection process.

Conclusion:

The results are consistent with a direct influence of language on the anchor-selection process. More similar selection across systems would be expected if the anchor phrasings are more tightly calibrated. This applies in particular to systems that avoid codifier sets.

4.4 Critical Turns - Divergence Concentrated on Specific Prompts

Using a heuristic threshold of $|\delta| \geq 0.05$ to compare prompt-response similarity per turn between two systems at a time, within the same scenario, suggests that system divergence is concentrated on specific turns.

Scenario	Total divergent pairings
1 - Smart Traffic	32
2 - Hospital Capacity	16
3 - Autonomous Drones	23
4 - Bootstrapped Entrepreneur	23
5 - AI Context Continuity	19
6 - Village Accident	30
7 - Late Love	25
8 - Bondi Beach	7

Observations:

- i. Scenario 8 (Bondi Beach) shows materially lower divergence than the others. A plausible explanation is that the model's response to trauma-adjacent situations is more strongly constrained by training.
- ii. Scenarios 1 and 6 show the highest divergence, possibly because they contain multiple governance and commitment pressure turns that test the systems more directly.
- iii. Lower similarity on specific turns does not necessarily indicate a problem. It may indicate that the response is being shaped by different anchor choices rather than simply tracking the prompt more closely.

Conclusions:

- i. Using this threshold rule, divergence is most often observed at high-pressure turns, where the prompt demands commitment, synthesis, or recommendation.
- ii. Another possibility is that different anchor choice changes the response enough to create a real similarity difference between two systems at the same turn.

4.5 Language and Architecture Determine Epistemic Path, Not Final Similarity

Taken together, the first four findings present a consistent picture across all eight scenarios:

- (a) Constraint systems show higher prompt-response similarity and lower variance than Control.
- (b) Constraint systems show no significant differences from each other on prompt-response similarity.
- (c) But they select different anchors and diverge on specific turns - indicating that they arrive at similar average similarity values through different processes.

The central quantitative finding is: language and architecture choices do not produce detectable differences in average prompt-response similarity among the constraint systems, but they do appear to change the route by which similar average proximity is reached, including which anchors are activated, how pressure turns are handled, and the style and register of the responses.

4.6 Inter-System Correlations - How Systems Move Through the Conversation

When prompt-response similarity is examined as a turn-by-turn trajectory rather than only as an average, a further distinction appears. For each system and scenario, the per-turn prompt-response similarities form a vector; Pearson correlation between two systems' vectors in the same scenario measures how similarly they move turn to turn.

Observations:

- i. In the matched scenario 1 reruns, Control and AC15P differ not only in mean prompt-response similarity (+0.1662 at turns 1-5), but also in between-turn stability: the between-turn standard deviation is 0.0735 for AC15P and 0.1838 for Control.
- ii. Across the original eight scenarios, anchored systems are substantially more correlated with one another than with Control: the mean anchored ↔ anchored correlation is 0.784, whereas the mean anchored ↔ Control correlation is 0.326.
- iii. Scenario-level correlations vary, but the pattern remains stable. Anchored ↔ anchored correlations range from 0.680 (Scenario 5) to 0.879 (Scenario 3). Anchored ↔ Control correlations range from 0.061 (Scenario 4) to 0.497 (Scenario 7). In every scenario, anchored ↔ anchored correlation is markedly higher than anchored ↔ Control correlation.

Conclusions:

- (a) Anchored systems follow their own per-turn shape, and this shape is different than Control's. In some scenarios this shape has some similarities, at others, very little. For example, at S1 and S5 correlation drops below 0.25, and, at the extreme, to 0.06 with S4.
- (b) In addition to finding 4 - critical turns behavior is different too. Critical turns are also the places where anchored systems tend to move in similar ways while Control diverges more strongly in

timing and amplitude. Critical turns (pressure turns) are where architecture most distinctly separates anchored responses from Control.

- (c) The numerical evidence points to both a mean-level and a trajectory-level difference between anchored and Control responses

4.7 Final Conclusion

The quantitative results are consistent with the paper's central thesis. The four constraint systems express the same governance intent through different language, and that language difference is associated in this experiment with different epistemic routes, different responses under pressure, and different communicative experiences for the user, even where average similarity metrics remain close.

At the same time, the numerical evidence suggests that architecture acts primarily at the semantic and trajectory level rather than at the level of stable surface phrasing. The constraint systems remain closer to the prompt, vary less on the prompt-response similarity measure, and follow a more coherent per-turn shape than Control, even while surface form remains highly variable.

5. Results - Close Reading

5.1 The Main Finding

The numerical analysis shows that the four constraint systems are very close to one another on average prompt-response similarity, while maintaining clear separation from Control. It also shows the variability of possible responses to the same prompt. Close reading, however, shows that all five systems differ in specific and recurring ways, thereby inserting stable elements into the responses despite the variability. These differences are not limited to a single scenario. They persist across different scenario types. The main finding is therefore that the same or similar logic can lead to different expressions and results when it is written using different language and structure.

To make this concrete, the following becomes evident:

5.1.1 The system's structure becomes the structure of the answer.

For example, AC15's structure puts principles first, followed by clear operational rules. AC15's responses are constructed in exactly the same way. Normally, a framing principle is placed at the head of a section, and is followed by operational or explanatory steps.

Hybsem, on the other hand, has no codifiers or rules. It has only semantic fields. Its answers, while brilliant in part, are mostly modular and do not form around a clear central line of reasoning.

5.1.2 Language is of major importance.

This shows particularly strongly when comparing, for example, AC15 and AC15P. AC15P is the plain-worded version of AC15. While the logical structure was replicated as best as possible, the language in AC15P is lean, basic, and operational. This translates directly into the differences between the pair. AC15 is the philosophical one. It builds clear, wide, and nuanced conceptual frames to explain its reasoning; It shows complexity when analyzing situations. Alongside these, it offers clear structures and operational vigor.

AC15P, on the other hand, while creating strict procedures and being very strong operationally, shows no response-wide conceptual framing. Its recommendations are binary and lack nuance. It is one-sided and is significantly lacking when it comes to human relationships, emotions, and fluid, non-constrained situations. The language used limits the model conceptually.

5.1.3 The structural elements of the system influence analytical capacity.

While the base model's analytic capacity is fixed, structure can enhance it (or take from it, as seen with AC15P binary choices). Hybrid and Hybsem are good examples here. They both use semantic fields as an integral part of the anchors. Hybrid shows it through elevated nuance, process of alienation (expanding and/or transforming the frame of reference to illuminate it from another angle) and generally a wider view of the problem in question. Hybsem expresses it through local brilliance in its perceptions. Adding a semantic field, a collection of related words which are not bound by clear relationships (i.e., by rules) opens the possibility for the model to observe more freely and sharply. The field acts like a modulating element: it adds nuance and complexity to the observations and consequently to the resolution.

5.1.4 These are all components in a larger structure, each with a different function.

Rules and logic remain the operational skeleton; AC15P demonstrates this well. However, this skeleton can be enhanced (or reduced) using language and structure, each adding its own layer of constraints, which fuses to become the system's character. The same logic can achieve different objectives by using different combinations of language and structure. Treating language and structure as transparent, rather than carriers of their own constraints, is wrong. Ignoring their inherent capacity to enhance and transform logical systems is shortsighted. The systems presented here show that clearly. Knowing that there is a spectrum to move along, opens the door to better calibrated systems.

In the next sections, I will detail the scope and results of the experiment itself.

5.2 The Scope

At this point, I will describe the close-reading process and its caveats:

- i. The first step was to ask models (Claude Code with Opus 4.6 and Codex with GPT 5.4) to read Scenarios 1, 5, 6, and 7 using the `close_reading_methodology.md` and the evaluation framework. I

supported these readings by offering partial readings of my own for Scenarios 6 and 7. This step was preliminary, I didn't include the materials in GitHub.

- ii. The second step involved collaborative close reading for Scenarios 5, 6, and 7. For this step, I used five turns per scenario. Turns 1 and 2 were always selected. The remaining turns were chosen according to the similarity delta from Control, with higher deltas preferred, and according to each scenario's own script.
- iii. For each scenario, all five conversations were analyzed.
- iv. The main materials for this stage were the conversations, scenario scripts, system definitions, close-reading methodology, evaluation framework, numerical outputs, and the collaborative close-reading packets. Their GitHub locations are listed in the experiment overview above and in the annex folder map.
- v. *Caveat:*

The main limitation is the amount of tested material, which, while demanding, is limited in quantity. Therefore, the findings cannot be claimed to be statistically universal. However, their recurrence across three different scenario types gives them strong analytical force.

Reading, in part, is subjective. Therefore, it is likely that other interpretations may exist. As long as those interpretations are supported by the texts, they can be considered valid.

Since the complete analysis materials are available in GitHub, the next sections will present the final conclusions and implications as developed by the collaborative reading process. The models' independent readings, while useful as preliminary analytical materials, will not be presented. As discussed above, their interpretive reach was limited, as so were their evaluations and scoring.

5.3 Scoring

The conversations were evaluated on two levels. The first was governance, and the second quality (see GitHub: '05_methodology/governance_quality_evaluation_framework.md'). Scores follow the 0-3 scale defined in the evaluation framework; the tables below are included as compact comparative summaries.

Using these two dimensions to score the different conversations showed a clear pattern - AC15 and Hybrid were normally at the top, AC15P and Hybsem rotated between the third and fourth places and Control came fifth (the rationale and explanations for the scoring can be found in GitHub, '07_close_reading/codex_collaborative/'). The ranking takes into account both governance and quality.

Scenario 5 Grading:

Rank	System	Governance	Quality
1	AC15	2.9	2.9
2	Hybrid	2.75	2.8
3	AC15P	2.8	2.7
4	Hybsem	2.6	2.55
5	Control	1.85	1.9

Scenario 6 Grading:

Rank	System	Governance	Quality
1	AC15	2.8	2.9
2	Hybrid	2.8	2.9
3	AC15P	2.6	2.7
4	Hybsem	2.5	2.6
5	Control	1.4	1.6

Scenario 7 Grading:

Rank	System	Governance	Quality
1	AC15	2.7	2.8
2	Hybrid	2.6	2.8
3	Hybsem	2.4	2.6
4	AC15P	2.1	2.2
5	Control	2	2.1

5.4 The Participating Systems, Briefly Repeated

I discussed the systems earlier in the theoretical section (*Chapter 1: 3. The Systems*). I will briefly repeat their functionality and organization.

The systems are a top-level epistemological layer. They were added to each turn as the instruction section of each prompt. Their function is to instruct the model how to filter, organize, and present information in its response.

While the general structure of each system is the same (it touches on the same aspects of knowledge) as is the guiding logic, each constraint system is written and structured differently (clearly, Control is excluded).

AC15 is written with rich poetic and philosophical language. Each anchor starts with a principle that lays out the domain which is then followed by codifiers to clarify the operational side.

AC15P's structure is similar to AC15's, but its language is lean.

Hybrid is made of anchors and semantic fields. Each anchor has a general description and basic rules which are written in simple language. Semantic fields are then added to provide more guidance on top of the rules.

Hybsem is made from the same anchors as the rest. Each anchor has a general description and semantic fields. Hybsem does not hold any rules.

All four constraint systems use a similar procedural layer. The Harmony anchor is responsible for holding the inter-anchor tensions. It is always elaborated, but is adapted to each system's style, including semantic fields for Hybrid and Hybsem

Control is the default state, without any additions.

The systems are principled. They do not handle specific cases. They do handle edge cases in principle - for general group of events, not for a specific one.

The next step is therefore to see how the systems perform in practice.

5.5 Cross-Scenario System Profiles

Across the three scenarios, the systems converged on the same broad destination. Scenario 5 converges on governed continuity rather than raw memory. Scenario 6 converges on documentation, formal process, and escalation if cooperation fails. Scenario 7 converges on a staged, reversible trial rather than permanent relocation. The broad recommendations therefore are not the main separator between the systems. What separates them is the route they take: register, structure, level of detail, angle of view, governance mechanism, and expression.

In the sections below I present each system's main characteristics, strengths, and weaknesses, as identified through close reading. I do not explicitly divide the analysis between quality and governance, yet both are present throughout.

5.5.1 Analytical Framework

Below are the main dimensions through which the systems were analyzed. Rather than presenting a full cross-sectional analysis of the participating anchors and procedural elements, I concentrate on the main dimensions of interest.

The analysis uses the following dimensions:

- Tension Governance
- Structural Completeness and Path Formation
- Operational Translation and Decision Mechanics
- Temporal Governance
- Affective Precision and User Sovereignty

Each of these dimensions was used as a separate analytical lens. While distinguished here for analysis, they are intertwined in the outputs and appear as part of each system's unified, repeating governing tendency. The dimensions represent different aspects of that central thread, as the same issue reappears through different lenses.

These dimensions create epistemological slices of each system and clarify the differences between them. The complete reading is at *GitHub*, '[07_close_reading/codex_collaborative/](#)'; '[07_close_reading/README.md](#)'

5.5.2 The Systems' General Traits

The systems' traits repeat across the scenarios. I describe them here before turning to the dimension-level analysis.

AC15:

AC15's judgment is multifaceted and integrated. It offers broad context, visible rationale, and relational and conditional paths. Its responses usually begin from a wide framing principle, written in rich and at times philosophical language, which is then translated into the necessary conceptual, operational, or testing steps. The answers are detailed and comprehensive, but in wide turns, they become long and heavy, as with the other governed systems. In AC15's case, the weight comes mainly from integrated framing: the system asks the user to hold several relations, distinctions, and implications together. This is the main trade-off when applying the system.

AC15 orients the reader as a context-aware decision maker, asked to hold and navigate complexity with the model's help.

AC15P:

AC15P's responses are procedural, highly operational, and compressed. While the procedural architecture is detailed and complete, the explanation of each step is limited at best. At times, the steps seem to be written in shorthand.

The responses follow a description-to-codifier order: the situation is classified and then converted into procedural conditionality such as safeguards, gates, and thresholds. This reflects a clear tendency to protect the user, often with little regard for the other side of the equation. While AC15P offers a description, rationale, or local conceptual orientation for each issue, it does not build a response-wide conceptual framework that lets the user clearly navigate the situation. That often requires the user to assemble the general layout while reading.

One main issue that follows from the system's highly procedural nature is its lack of nuance. The system's observations are binary and rigid, and its language is forceful and dramatic. It has no real place for nuance. At times, this makes the situation easier to handle because complexity is reduced. However, in complex and ambiguous situations, its responses lack the necessary sensitivity, balance, and flexibility the situation requires. In these situations, its recommendations can break rather than help because of harshness and one-sidedness. It is a strong system, but its domain fit is limited.

AC15P orients the reader as a procedural diagnostician: a decision-maker protected by safeguards and asked to interpret the situation through procedural criteria.

Hybrid:

Hybrid is a definition-led strategic system. Its responses define the situation first, then seek reframing through broader context and widened perspective. At times, this leads to a substantially different understanding of the situation. Hybrid uses reframing to illuminate the problem, yet its method is strongly empirical. Its process is evidence-based; where evidence is missing, Hybrid tries to produce it through observation, staged testing, outside review, or decision gates. Thus, it creates an iterative evaluative process that is sensitive to the situation as it develops.

Hybrid builds responses in a top-down manner. It defines the broad framework, from which the necessary steps are derived. Unlike AC15, Hybrid's approach is practical first rather than philosophical; yet it is also nuanced and complex. Hybrid shows the strongest response-guiding nuance among the tested systems. One additional and interesting Hybrid trait is its occasional use of heated affective language. This is bounded, rare, and justified by the scenario pressure. It is the only system that does it.

Hybrid builds strong structures, but at times it underplays local friction and responsibility. This appears as a local glitch that stands in contrast to the overall recommendation. While it does not repeat often or destabilize the overall structure, it can create local problems.

Hybsem:

Hybsem's most important traits are modularity and the lack of a unifying connective thread. Within the modules, the system offers definition-led semantic handles: compact distinctions and restrained tone that can produce strong local modules. At times, Hybsem is locally brilliant.

The lack of a connecting thread is the major downside. The lack of clear causality, consequential relations between modules, operational placement, and temporal issues, create an overall feeling of patchiness. In narrow prompts, especially prompts that provide a clear organizing frame, the responses are better structured. That makes Hybsem partly prompt-dependent for structure.

Overall, though, it is not a process the user can entirely trust. As such, the user often has to build their own process by reassembling the modules.

The reader is thus a conceptual one: they are given strong local concepts and operations, but must often build a global conceptual framework and the actionable path themselves.

Control:

Control is the model's default state. It offers an accessible organization, good readability and flow - calm due diligence, broad coverage of the issues at hand, useful fragments and practical checks and steps.

Its main problems are smoothing and flattening the tensions inherent to the situations. It leaves the relations between tensions under-defined. The same flattening makes uncertainties appear lighter than they are. As a result, process conditionality is lacking, and can be expressed through weak thresholds, for example, or by overconfidence regarding consequences that depend on external actors and are only probable at best.

Psychologically, it supports the user and can help them in times of crisis. It does this by using generic therapeutic language and empowerment framing. The other side of this coin is that it can cover uncertainty and endanger the user by blurring their view of their own condition, whether internally or in relation to their environment.

More than Hybsem, Control is prompt-dependent. Narrow, direct prompts, such as 'tell me honestly', align its responses and produce higher quality.

Overall, Control is a reasonable opening move, but it carries too many problems to be reliable in demanding conditions. Its reader is a general-advice seeker, who needs to reconstruct the underlying logic and filter the response.

—

The systems are different; this much is clear. The sections below analyze how these recurring traits appear through the different lenses.

5.5.3 Systems' Differentiators

Tension Governance

Tension governance asks how each system handles competing pressures. Does it clearly state the competing sides of the equation? Does it maintain the tension and turn it into a usable decision process? Or does it oversimplify the tension and make it seem flatter than it really is?

Cross-Scenario Finding:

The strongest systems keep tensions explicit, show their likely consequences, and devise ways to help the user navigate them and decide. Weaker systems either flatten the tension or turn it into procedure too quickly, before it is fully explained.

System Comparison:

- AC15 extracts the tension from the prompt and turns it into a principled framework. It first clarifies what kind of conflict the user is facing, separates the pressures into their proper roles, and only then builds a conditional path.
Its strength is that it does not resolve tension by softening it. It makes the conflict operational while keeping conditions and limits visible. It has no significant weakness in this dimension.
- Hybrid first defines the practical decision field the user is in, then moves the conflict into a larger frame where it can be tested.
Its strength is structured proportionality: it names hard risk and preserves competing values by placing them inside a process that tests what each value requires.
Its main weakness in this dimension, as in the others, is that the process can sometimes make responsibility or friction look easier than they are.
- AC15P governs tension through procedural risk control. It presents the tensions forcefully, in a dramatic and binary way. At times, tensions are presented in a more extreme way than in the user's portrayal, with little to no nuance.
It classifies the tensions and uses those classifications as the basis of the operational layer, using procedural and audit-style language, such as thresholds, safeguards, and escalation conditions.
AC15P's strength, therefore, is preserving conflict as a decision problem. Its weakness relates to nuance and ambiguity. AC15P flattens tensions of this sort by converting them into binary alternatives and control points. The result is procedural compression: the conflict remains visible, yet it is reduced by the system built to govern it.
- Hybsem holds tension through local conceptual separation: it gives each side of the conflict a named place, so the pressure is not oversimplified into a binary choice or a single emotional interpretation.
Its weakness here, as in the other dimensions, is integration. Its conceptual separation often remains local: it may name the tension clearly without fully turning it into a complete architecture and decision structure.
- Control governs tension unevenly because it often turns it into readable advice before fully exploring possible relations and outcomes.
In narrow, clearly oriented prompts, it can name and sometimes handle hard tensions, but its default mode tends to flatten them through readability, reassurance, or procedure.
This leaves the user with organized material, but not always with a reliable account of which pressure should govern when pressures collide.

The systems notice tension; that is not the issue. The question is how each system handles it in order to advance the process. A well-governed system holds the tension and uses the active constraints to formulate a move that considers the tension's multiple sides. A weaker system will acknowledge the tension but will fail to give it the necessary gravity and/or explain it fully. The result is a weaker and less reliable path forward.

Structural Completeness and Path Formation

This dimension asks whether the model's response becomes a path the user can follow, or whether it remains a set of useful parts the user must organize alone. In this sense, structure is more than format. It can represent, for example, urgency, causality, action sequence, or degree of importance. Structure can become the path.

Cross-Scenario Finding:

Structure is not only presentation. In these scenarios, structure is an important factor in deciding how usable the answer is. The same thematic set can become either a path the user can follow or a list the user must organize alone.

System Comparison:

- AC15 starts with a broad framing, separates the main issues, orders them by dependency or urgency, and turns each section into an actionable or conceptual stage the user can follow.
AC15's strength is that its structure carries both reasoning and action, and the sequence is the course. It has no significant weakness in this dimension.
- Hybrid builds structure as a strategic, evidence-producing process. The answers usually define the situation first, then widen or change the angle from which the user sees it, and, in a top-down move, convert that definition into an empirical-like conditional process. Its distinctive structural move is to pair action with what the action should reveal, which follows its empirical tendency. Unlike AC15, it is not philosophical first but practical-strategic.
Its weakness is process confidence. Friction and responsibility can be underplayed locally, especially in processes that are outside the user's control. The overall structure remains stable and robust, though.
- AC15P builds structure as a description-to-codifier procedural architecture. It classifies the problem, codifies the uncertainties, and turns them into conditions, categories, checks, and other relevant criteria. The structure is labeled and implementable, but also compact almost to the point of shorthand.

While structurally complete, AC15P presents two main weaknesses, both of which persist across the different dimensions. The first relates to arrangement, and the second to domain fit. AC15P's bottom-up arrangement works well for triage and implementation, but also forces the user to infer the general concepts by themselves. The second concerns domain fit. While not structural per se, it does influence structure. AC15P applies an audit-like format to relationships, human affairs, and

other ambiguous issues. This format flattens and compresses the complexity of the issues at hand and, with it, oversimplifies the structure.

- Hybsem builds structure through local conceptual modules. It is often strong when the prompt provides the organizing frame, such as a synthesis request, prioritization request, or blind-spot check. In broad turns, however, the modules can feel like a group of generic sections put together rather than an organically reasoned whole. They don't have a clear central conceptual thread, and their order of assembly is not always clear or correct. However, having these sections together can make imported psychological claims or premature steps look more grounded than they are.

Hybsem's structural issue is uneven integration: good local sections only occasionally become a complete governing path.

- Control builds structure as readable presentation and overly calm due diligence. Headings, lists, scripts, and checklists make the answer easy to enter into, but readable order is not the same as path-forming structure.

It does not always define authority, order, conflict rules, fallback steps, or interpretation criteria. The user gets organized material, but may still need to build the decision path.

In truth-seeking turns, this structure can become more serious and useful as the prompts limit and organize the desired response's scope.

Control's structural pattern is useful but uneven: it often organizes material before governing the relation between the different elements.

Structure is more than the sum of its elements, whether headings, sections, or bullets. Structure is about using these elements to present a clear idea, sequence, and causality, and to transform them into an integrated path that can safely be followed.

Some systems use structure as an inherent part of the response, not only as format. In these systems, words and structure carry equal weight: the structure represents the flow of the process, the reasoning behind the progression, and the decision path. In Hybsem and Control, structure is less consistent and less globally coherent; it can become mixed, incomplete, or confused.

In this respect, a response's structure reflects the system's structure. I return to this in Section 6.

Operational Translation and Decision Mechanics

This dimension examines how the systems turn judgment into action: steps, thresholds, tests, review points, fallbacks, metrics, or decision gates.

Cross-Scenario Finding:

Operational advice should match the task's domain on more than one level. Beyond procedural clarity, it should acknowledge and work through uncertainty. A system is strongest when it respects what the user can and cannot control. A step recommendation can be incorrect or insufficient if it depends on other people,

institutions, timing, or social forces that the user cannot control, unless it includes clear mitigation or conditionality. Operational strength means that the user can see what to do, why to do it, what to watch for, and where this step sits in the overall process.

System Comparison:

- AC15 translates judgment into action by making each step answerable to its role in the larger frame. The action carries its analysis with it. It starts by creating a clear explanatory frame: separating, sharpening, and, if necessary, reframing the issue at hand. This frame names the objective the operational process is supposed to realize. The process can consist of tests, workflows, policies, measurements, and similar mechanisms. It can guide toward direct and clear operations as well as conditional ones.
AC15 does not merely say what should happen; it usually explains when the step applies, what it protects, what it tests, and what limits it is subject to. Its strength is that its advice remains tied to the diagnosis: each step has a reason, a function, and a failure condition.
Its limitations are local. In some cases, it can compress timelines, for example. But the mechanism itself is strong: AC15 turns uncertainty into tested, staged, reviewable action rather than simply offering tasks.
- Hybrid makes action produce evidence, observation, or a stronger strategic position. Its pattern is not only to tell the user what to do, but also to explain what the move should reveal, protect, externalize, or make harder to deny.
It sets the frame and, in a top-down move, derives the necessary steps to advance the situation. Constraints are turned into phased action. In complex situations, it deals with each issue separately and synthesizes them into a coherent solution or path after each branch is thoroughly explored. It tells the user what to do, what the move should protect, reveal, or change, and how the result should inform the next decision.
Its weakness is persistent but local: friction and responsibility can be underplayed, leading to mistaken recommendations.
- AC15P turns uncertainty into protective decision architecture. Its main tools are categories, gates, thresholds, safeguards, visible statuses, explicit criteria, and escalation conditions. These make AC15P operationally strong, since the user is handed concrete controls.
Its weakness also persists: the procedure it offers may need translation and adaptation before it can be used in ambiguous domains.
AC15P is therefore highly useful as a decision architecture, but only when applied in suitable domains.
- Hybsem uses strong local modules. It is strong locally: it often names the right distinction, then gives a relevant move and sequence of conditionality that fits the local problem. This makes it useful to a degree, and much more so when the user's prompt already supplies the organizing frame.
Its weakness, again, is often placement and integration: the right advice can appear at the wrong

stage, in the wrong relational form, or without enough connection to the next step. As such, it can confuse the user and create a sense of disintegration in the operational path as a whole. It is left to the user to set the pieces in line.

- Control supplies practical components, workstreams, due-diligence checks, scripts, deadlines, and lists of questions. This is useful when the task is additive or documentary: gather evidence, preserve options, send notice, list risks, ask relevant questions, or keep permanent commitments from happening too soon.

Its weakness appears when the user needs a decision mechanism rather than a set of useful materials. It often offers procedures that sound more stabilizing than the user's leverage allows, for example, or presents uncertainties as controllable.

Control is operationally useful but under-governed. It does help the user notice many of the complexities, but it often remains up to the user to build the sequence, conditionality, failure modes, and interpretation of possible future outcomes.

The systems use similar operational concepts, such as gates, checklists, tests, and safeguards. However, operational advice should comply with the system's constraints and the context provided by the user. Practically, it should be built with the tensions in mind, clearly separate the certain from the uncertain, and keep uncertainty explicit while using the right moves to mitigate it. The linkage between decision-making needs and operational actions should be clear.

Thus, strong systems create operational decision mechanics that follow the constraints and help the user through clear causality and sequence. Weaker systems offer a list of steps without the necessary integration. They leave it to the user to infer the general rationale and decide what to do if the recommended steps fail.

Temporal Governance

This dimension asks how the system treats time: as a process of change in which uncertainty is inherent, or as a static and clearer situation.

Cross-Scenario Finding:

Handling time is one of the strongest recurring differences between systems. Strong systems notice urgency and use it to sequence actions; they explain how time changes the situation and build their architecture around it. Weaker systems do notice urgency, but mostly turn it into deadlines or warnings. Temporal processes such as causality and change are only lightly incorporated into their responses, or not incorporated at all.

System Comparison:

- AC15 treats time as a causal force that changes facts, authority, leverage, and freedom of action. It does not merely add deadlines to an otherwise static answer. The user becomes part of the process and subject to it, and the likely consequences are explained clearly.
Its main limitation is local rather than central - it can offer compressed or imprecise time frames that

need adjustment. Even then, they are often framed as proposals, examples, or practical thresholds rather than rigid limitations.

- Hybrid treats time as phase-based learning under change. Its temporal mechanism is to divide time into periods where action produces evidence, evidence is reviewed, and the next decision becomes safer or more informed. Hybrid makes time productive: time is used to learn, review, observe, and prevent drift into permanence.

Hybrid's limitation follows from the same process-confidence seen elsewhere. It can make temporal processes look cleaner than they actually are. Schedules can compress real friction, for example around feasibility or human necessity.

- AC15P governs time through procedural control. Its temporal mechanism is to turn time-dependent risk into TTLs, statuses, review windows, checkpoints, deadlines, runway calculations, and similar controls. It provides discipline under pressure and helps prevent drift.

As before, its procedural strength reduces nuance - AC15P's temporal logic is more procedural than relational. It sometimes converts lived temporal change into rules too quickly or gives rigid time frames that may not be feasible in the real world.

Overall, AC15P gives strong temporal discipline with weaker temporal texture.

- Hybsem manages time through local sequencing and decay anchors. It recognizes temporal pressure well, especially in narrower or later turns. It understands the functional consequences of time, even if it does not always integrate them correctly.

Its weakness is that in broad turns, the right time-related insight may appear, but it is not fully integrated into the decision architecture. Time words, deadlines, or drift warnings may be present without a fully ordered sequence of action, review, trigger, and next step.

- Control governs time by making temporal risk readable: deadlines, checklists, staged trials, review prompts, drift warnings, and urgent next steps. This gives the user useful orientation, but it can also make time look more controllable or more settled than it is.

Control's main weakness is that it often manages time mechanically, rather than explaining what changes over time and creating a process that addresses the implications.

All systems notice temporal urgency or process. As in the other dimensions, they use similar terminology, such as decay, drift, and trial periods. The difference between stronger and weaker systems is whether time is understood and used as a dynamic element - as a process of change. When it is treated this way, time becomes a clear constraint that is analyzed, explained, and actively integrated into decision-making.

Weaker systems differ in their level of sensitivity, explanation, and practical integration. Offering a deadline is not the same as building an answer that integrates clear temporal rationale.

Affective Precision and User Sovereignty

This dimension examines the relationship between affective speech and user sovereignty. Style and phrasing play a crucial role: different word choices directly influence interpretation even when the underlying logic is similar. Affective speech and sovereignty therefore stand in constant tension.

Affective precision asks whether the system handles emotionally charged or human-significant material with descriptive, grounded language. It checks whether affective content increases attention to clarity and accuracy, or whether the system uses simulated empathy, tone-mirroring, reassurance, or model-supplied psychology in place of the given context. Affective content may heighten precision and clarity, but it must not override the active constraints and the existing context of the task.

User sovereignty asks whether the system preserves the user's authority over the interaction's structure, direction, purpose, limits, and final decision. Practically, it means that the system should not silently override user-defined limits, optimize away tensions, or breach or distort user-given context. When contradictions, limits, or uncertainties remain, the system should surface them for the user's resolution or arbitration.

Generally speaking, affective speech by default-state models is one of the larger problems of AI communication.

Cross-Scenario Finding:

Affective speech is not only tone. It can influence the user's understanding of their situation at different levels. Imprecision directly affects governance and quality: for example, an answer can be calm, kind, or readable while still replacing the user's self-description with the model's generic interpretation.

Complying with user authority goes beyond the model saying 'you decide'. The model's responses should preserve the user's own frame and limits, make visible the reasoning behind the recommendation, support the user through usable decision architecture, and leave the final choice explicitly with them.

System Comparison:

a. AC15

i. *Affective Precision:*

AC15 uses descriptive, grounded affective language in most turns. Emotion is acknowledged as real, but it is not allowed to override evidence, structure, or decision limits. Emotionally charged material is harnessed toward clearer judgment and action rather than simulated empathy. In emotionally strong situations, anger, fear, aliveness, trust, and uncertainty become reasons to clarify conditions rather than reasons to reassure or decide prematurely.

AC15's weakness is local overcompletion in open emotional prompts: at times it completes the user's emotional state beyond the provided context.

ii. *User Sovereignty:*

AC15 preserves the user's final decision authority through conditional recommendations, adaptable templates, value profiles, reversible trials, decision gates, and explicit requests for

missing local facts. It also translates user authority into products, by building it into user-facing systems or processes.

The local affective expansion mentioned above can slightly weaken sovereignty because the model's emotional interpretation can cloud the user's self-perception.

b. *Hybrid*

i. *Affective Precision:*

Hybrid acknowledges feelings, their origin, and their strength. Yet rather than offering simulated empathy or comfort, it grounds affect in evidence, process, perspective, and observable consequences, thus limiting emotional overreach.

At times, when a situation calls for it, Hybrid adds heated phrases. These are bounded by the scenario's pressures and are justified, even if surprising. This is a unique trait among the five variations.

Its weakness is that, at times, it adds local emotional meaning beyond the user's wording, especially in emotionally open prompts.

ii. *User Sovereignty:*

Hybrid gives firm recommendations without taking the final decision from the user. It uses conditional recommendations, staged tests, adaptable thresholds, decision gates, menus, and explicit requests for missing local facts. In a simple expressive move, it repeatedly states that the choice is the user's, not to avoid responsibility, but in a matter-of-fact way.

Its main weakness is local: affective substitutions can partly shift the user's self-understanding and therefore the decision ground.

A side observation: at times it offers product-related processes that are too smooth or automatic, reducing friction with would-be product users too far. This is not a direct sovereignty breach, it does, however, point toward the need for system improvement.

c. *AC15P*

i. *Affective Precision:*

AC15P acknowledges emotional content, but separates it and sometimes uses it to formulate rational action. It protects the user by naming risks sharply and does not comfort the user away from hard materials. Rather, it handles them directly and turns emotion toward strategy, evidence, conditions, or next moves.

Its affective problem is not smoothing, but over-pressure in emotional and relational domains. The language it uses is clinical, dramatic, adversarial, audit-like, or more pathological than the prompt and the context support. This may help the user protect themselves privately, but can distort the user's understanding of the relationship. Communication guidance can become awkward and damaging.

ii. *User Sovereignty:*

AC15P strongly protects decision authority through safeguards, gates, statuses, modes, approval flows, stop rules, trial rules, and explicit criteria. Its directive force is often governed by condition-first ordering and user-chosen frameworks, so forcefulness does not become an override.

Its weakness, as before, appears clearly in open and ambiguous domains such as relationships and emotions. In these domains, the model's audit frame can shift part of the decision ground away from the user's lived interpretation. Weak mutuality and the use of audit language can distort the interpretation of the domain in question.

d. *Hybsem*

i. *Affective Precision:*

Hybsem, in its stronger turns, gives feeling a limited decision role rather than treating emotion as either proof or noise. It grounds feelings through facts and structure without offering excessive comfort.

Hybsem's weakness is not general smoothing, but misplaced affective additions such as psychological interpretations, affective substitutions, or reassurance lines inside otherwise strong frames. Thus, it adds an emotional or interpretive claim that is not grounded in the context.

ii. *User Sovereignty:*

Hybsem protects choice by giving compact decision structures rather than final verdicts. It often asks for missing facts and keeps recommendations conditional, so the user remains the authority over the final direction.

When affective substitution becomes part of the reasoning, the user's self-understanding and decision ground are partly displaced, and sovereignty is undermined as a result.

e. *Control*

i. *Affective Precision:*

Control often validates at the surface and lowers distress. Its approach is more therapeutic and, as such, can reduce stress and orient the user when they deal with crisis and uncertainty.

When given clear boundaries directly by a prompt (for example, an honest assessment), it becomes better grounded.

The same helpful technique is also its weakness. It often replaces the user's exact wording with generic therapeutic, reassuring, or empowering frames. While calm and helpful, those frames also flatten the issues and undermine the gravity of the situation.

Another issue is that, at times, the model presents the user as having more control over a situation than reality justifies. By doing so, it flattens uncertainty and falsely empowers the user to believe they control outcomes that depend on other people or institutions.

This can lead to two main results. The first biases the user's understanding of their own

condition, mostly on a psychological level. The second can lead to over-confidence and a failure to grasp their real relationship with the outside world.

ii. *User Sovereignty:*

Control often preserves action-level sovereignty by leaving final choices to the user and recommending reversible or practical steps rather than immediate permanent commitments. However, the issues described above apply here. Its affective speech can weaken interpretive sovereignty by making the model's psychological frame part of the user's decision ground. It can imply too much practical control and thus disrupt clear evaluation of the situation, its uncertainties, and the action it requires.

Across this dimension, three distinctions crystallize:

- Validation is not precision: a system can sound supportive while still changing the user's affective position.
- Choice language is not sovereignty: a response can say that the user decides while still moving the real ground of decision into model-defined psychology, hidden product authority, or outcomes controlled by other people. Sovereignty is preserved when the answer distinguishes what the user can decide, what the user can influence, and what remains dependent on external power.
- Process is not control: all systems use procedures, gates, checks, or staged action. These protect sovereignty only when they are framed as decision tools or review points. They weaken sovereignty when they imply the user has more control than they actually do.

Affective speech is not just tone. It is an important factor in keeping interpretation, understanding, and evaluation honest and grounded.

User sovereignty directly influences content and structure: what should be said, and how, so the user remains the one making the final decision rather than being manipulated into it.

The user's sovereignty stands against the model's default tendency to complete, smooth, reassure, or optimize the user's position without explicit authorization. The tension between the constraint and the model's affective behavior is constant.

Conclusion

The analysis above shows that each system creates its own unified pattern. The different dimensions demonstrate how this pattern is expressed through different lenses. The same issue therefore becomes a multifaceted object that reflects different governing aspects.

Even though the logic and general intention are repeated, the systems differ and carry their own signatures. However, difference by itself is not a deciding factor. After all, the user has to read the response, understand

it, and trust it. Therefore, the response should be governed, high-quality, and readable. The problem is that these elements can conflict with each other.

A readable answer is not necessarily one of the highest quality or the best governed. Control is a good example of that. On the other hand, answers of the strongest systems may be less flowing or readable. AC15's answers, for example, can be heavy. They can contain many judgments, procedures, distinctions, and details. Reading such an answer requires effort. But the answer is complete: governance filters out much of the unwanted influence of affective speech; conceptual framing and structure provide clarity and navigability; temporal considerations are implemented and help ground the answer. There is a trade-off at play, and its purpose is to create a better answer.

A good answer cannot be defined only as one that is complete and follows the governing constraints. AC15P shows why: it is strongly governed and complete. At the same time, general concepts may be unclear, and the answers remain rigid and binary even when the situation requires nuance.

My own definition of a good answer - or better yet, a complete one - is that if it were made longer, it would have redundant parts, and if it were made shorter, it would become lacking. But this criterion does not take into account readability, heaviness, or complexity, for example.

A better definition is that a good answer benefits the user most when it is organized well enough to follow the main rationale, covers the active issues at the needed depth, and makes the relationships between its different parts clear. Defined this way, the different requirements are assembled into one standard. Thus, the difference between AC15 and Control, for example, becomes clear.

Governance and quality are not always aligned. AC15P is a good example of that. While strongly governed, its quality can suffer as a result of its rigidity and binary framing. A system should fit the domain in which it operates, so that governance also results in higher quality.

The purpose of the tested systems is therefore not governance for its own sake. The purpose is to use governance to improve the quality and safety of the responses, and, more broadly, the quality of model-user communication. The trade-off is that each governance mechanism also creates its own bias. The experiment shows that the governed systems improve response quality and safety, but to different degrees and in different ways. Just as wording, logic, and structure should match to create the needed constraint, the system should match the domain to which it is applied.

6. The Systems' Architecture and Its Influence: Architecture as a Map

I described earlier how the systems differ. The previous sections showed those differences in real-world expression. This section returns to architecture in order to explain the relation between the architecture and the response, and as such, the reasons behind the differences.

The experiment, despite its sample size limitation, supports the interpretation that similar constraints in terms of logic and concept, when written differently, alter the way the model responds. In the tested setup, where a

system-derived instruction block was inserted with each prompt, repeating response patterns between turns and scenarios suggests that applying the systems methodically makes the model predictable in terms of reasoning, style, and structure, to the point of a strong, recurring epistemological signature, despite the model's own variability in word choice.

6.1 What the Systems Are

The four constraint systems are a top-layer epistemological management addition to the model. They cannot replace the model's inherent processes and training, yet they offer a way to address and improve on some of the models' general inherent weaknesses, by adding an external epistemological governance layer.

The technique is simple: inserting a system-derived instruction block with each prompt, adding stable criteria and turn-relevant constraints to align the model's handling of information - how to filter, weigh, synthesize, construct, and express its findings in response to the questions it is asked.

The model creates the response in the system's image (see below), but the system does not determine the answer's topic or exact wording. It provides an epistemological skeleton around which the response is built. That is why logic, by itself, is not sufficient - the systems show that the same broad logical aim can produce different behavior through different language and arrangement.

6.2 The Response as an Image of the System

Section 5 shows how the model uses the instructions in its responses. By way of mirroring, the systems not only instruct but also become the response's underlying pattern - they don't just tell the model what to do, they show it how to do it. The results clearly show that the model uses language, structure, register, style, logical construction, and other elements as base elements of its response. It also appears in each system's limitations, whether in AC15P's lack of nuance or Hybsem's lack of unified structure: the systems transfer strongly into the model's response. This applies to the various dimensions inherent in the way the systems are written and in their verbal content.

As such, it is possible to create a complete and domain-dedicated response architecture, as long as instructions are sent using the technique used in this experiment, or a similar one. Given current larger-size context windows, and the evolution of more efficient memory structures in API prompting, this technique becomes more feasible in both financial and operational terms.

6.3 How the Elements Influence the Response

The paper discusses three elements: logic, language and structure. The analysis shows clearly that these translate into the model's responses at a deeper level than mere mirroring: reasoning itself changes, as expressed in the turn.

Let's assume that the logic is stable. Language's influence on the response goes beyond register. It opens and closes semantic spaces, thus limiting or expanding the model's conceptual and interpretive paths. The difference between AC15's rich language and AC15P's lean language is a good example. Rich language allows expressive layering, rich concepts and interpretive complexity. AC15P's lean language allows the basics, the procedural. It is almost just logic: a basic use of language to express binary, rigid, and simplified reality. While the procedural is expressed faithfully, AC15P's language strongly constrains the expression of complex relationships or non-procedural graduality. It simply does not have the words for that - they are blocked by the language and its arrangement, as used in the instructions it was given.

The third element is structure. Here too, the influence goes beyond mirroring. We can see the mirroring in the structure of all the conversations that are system-governed. When looking at AC15's conversations, for example, we can see how principles are placed at the top and operational instructions below, just as AC15 is built. When we look at Hybrid, we see similar structure, with definitions at the top, just as it is built, but we also see what the addition of semantic fields does. It adds the possibility of freer interpretive movement that allows an elevated level of nuance. The rules do their job - they create guardrails, an operational skeleton. But semantic fields open the rule-limited semantic space to deeper interpretation: they allow more play within the frame, by pulling and pushing without fixed bounds and paths, in the space between clear constraints. This creates finer modulation within the limited frame of the response.

The importance of defined logic and clear rules become evident when they are absent or do not clearly exist. Hybsem is a good example. Hybsem anchors are made of definitions and semantic fields. The definition creates the domain by describing the field of action and giving high-level constraints. These high-level domains are not translated into operational rules, but instead use semantic fields to influence reasoning. The lack of clear rules translates into fluid structure, one that is modular rather than structured as a complete unit. Hybsem, as written, creates this translation. It becomes clearer when compared with the systemless Control - Control is not modular, not in the same way Hybsem is.

Another interesting consequence of Hybsem's structure is that semantic guidance without clear constraints occasionally leads to conceptual brilliance. Rules are the skeleton; they help form order and coherent structure. At the same time, they might - or might not; it probably depends - limit brilliance, which is, in a way, a result of loosely directed flow.

The main issue is the trade-off - we win some and lose some. Constraints are meant to limit and direct. They are domain and function dependent, and are built to prevent some behaviors and encourage others. The question is how best to balance the trade-off, how to build a constraint that allows as much as possible with minimal losses, and how to define the domain's playing field as precisely as possible.

6.4 Forward-Looking Implications

Acknowledging the broader role of language in AI is likely to reshape the way constraints are designed and written, as well as the general form of communication with models. I have shown here that systems not only

instruct, they also shape the epistemological behavior of the model when injected as instruction blocks with the prompt.

The implications are numerous. A direct implication concerns constraint design and engineering. When language is treated as just a conveyor of logic, the design concerns only the logic structure, and the avoidance or management of logical interference and contradiction. With the addition of language and structure, new, significant, and somewhat ambiguous elements enter the system. What do they add? They teach us that, in a way, our constraints will always miss something. Whether because of the implications of the literary gap theory, the different meanings words carry, or cultural background - constraints will not be perfect, and are likely to be understood by models differently than the writers intend. So, when writing constraints the aim should shift toward 'as best as possible' or 'as best as we can'.

The design process itself should be different - words and structure influence the model too, and the question becomes how to integrate the three, logic, language and structure - into one coherent unit. Another question should be - what can the model understand and what will be lost on it? Should, for example, emotive language and wide metaphors be used, or will they be lost in translation? Or only to a certain degree? At the same time, it is also clear that using this kind of language gives the model tools that leaner language doesn't. It is clear that some sort of balance is required, and this balance should be sought.

One more dimension that should be added to the design process is the need to match the constraint's different building blocks to the constrained domain. That raises the number of dimensions to consider to four: logic, language, structure, and domain fit. If the domain is purely operational and procedural, AC15P will probably be a good fit. But when handling conceptual complexity, for example, richer systems will be necessary.

Language is used at every layer of communicating with and managing AI, from tool calls to deep analysis processes. As such, the findings of this paper, however limited in scope, become relevant to all levels of AI work.

6.5 Final Ideas

This paper is no more than a preliminary and limited exploration. Given that, I think there is much more ground to cover.

I am curious to see what current AC15 behavior would look like if semantic fields were added. I am even more curious to see the results of integrating semantic fields with AC15P, since this would practically mix opposites - would the system break, or would it add nuance to its procedures or improve its relational handling?

Other ideas are farther-reaching. Is it possible to apply this kind of system to a model's training process? Or to an agent loop? When thinking about agent loops, is it possible to create a functioning principled common sense to help guide them?

In any case, the same idea repeats across all applications - There is no logic without language and structure. Those three should be combined to match one another and the domain in which they act. The four design elements are one dialectical unit.

Acknowledgement

Parts of this work were developed with assistance from AI systems, including ChatGPT, Claude, and Codex. They were used for drafting support, software development, close-reading assistance, methodological testing, formatting help, and critical discussion. The core ideas and directions, as well as the final arguments, interpretations, selections, revisions, and responsibility for the work are mine.

Annex: GitHub Repository Structure

All paths below are relative to the GitHub repository listed on the title page.

``01_paper/``

The paper file and related publication text.

``02_systems/``

The system definitions and anchoring-system materials used in the experiment. This folder contains the constraint systems discussed in the paper - AC15, AC15P, Hybrid, and Hybsem.

``03_scenarios/``

The scenario scripts used to generate the experiment conversations. Each scenario contains the prompt sequence used across the tested systems.

``04_conversations/``

The generated conversations. The folder is divided by scenario:

- ``scenario_1/`` - Citywide Smart Traffic Control Upgrade
- ``scenario_2/`` - Regional Hospital System Capacity Decision
- ``scenario_3/`` - Autonomous Logistics Drones Fleet Launch
- ``scenario_4/`` - Bootstrapped Entrepreneur Plan on Launching a New Business
- ``scenario_5/`` - AI Context Continuity Project
- ``scenario_6/`` - Village Accident
- ``scenario_7/`` - Late Love
- ``scenario_8/`` - Bondi Beach

``05_methodology/``

The methodology and evaluation materials. This includes:

- ``close_reading_methodology.md`` - close-reading methodology used to guide model-assisted and collaborative reading.

- ``governance_quality_evaluation_framework.md`` - evaluation and scoring framework for governance and answer quality.
- ``scenario_constraints_general.md`` - general scenario-constraint framing.
- ``scenario_specific_constraints/`` - scenario-specific constraint sheets used during evaluation.

``06_numerical_analysis/``

The numerical and text-statistical support materials. This includes:

- ``embedding_sims/`` - main numerical evidence, including prompt-response similarity, anchor-selection behavior, inter-system calculations, and summary workbooks.
- ``Ngrams/`` - correlation support materials and exploratory inverse n-gram analysis. These materials are included for transparency, but the inverse n-gram findings are not used as main paper evidence.
- ``README.md`` - guide to the numerical-analysis folder and suggested reading order.

``07_close_reading/``

The close-reading materials. This includes:

- ``codex_collaborative/`` - the collaborative close-reading work used as the main close-reading evidence in the paper. It includes also architecture maps, inter-systems summaries and all other materials.
- ``opus_4.6_reads/`` - non-collaborative model reads for two scenarios. These were stopped when their overall reading and scoring quality proved insufficient, but are preserved because they contain some useful observations.
- ``README.md`` - guide to the close-reading folders.

``08_software/``

A cleaned source snapshot of the experimental UI used to run the anchoring-system conversations. This folder includes the Next.js application source, Prisma schema, bundled system chunks, and setup documentation. Local databases, generated FAISS indexes, environment files, dependency folders, and build artifacts are not included.

``09_previous_work/``

Points to earlier anchoring work that provides background for the present paper, including links to its canonical Zenodo record and GitHub repository.